Krish Gupta

Intern at SafeQbit

Log Book @ SafeQbit

Introduction

1.1 Background

With the rapid growth of quantum computing, many of the cryptographic algorithms used today are expected to become insecure in the future. Algorithms like RSA and ECC are currently used almost everywhere, including websites, banking systems, digital certificates, and secure communication protocols. These algorithms are considered secure against classical computers, but they can be broken by a sufficiently powerful quantum computer using **Shor's Algorithm**.

Because of this, researchers have started developing **Post-Quantum Cryptography (PQC)**, which consists of algorithms that are believed to be secure against both classical and quantum computers. Recently, NIST standardized several post-quantum algorithms that can be used for encryption and digital signatures.

However, replacing the cryptographic algorithms alone is not enough. The complete security infrastructure, especially **Public Key Infrastructure (PKI)**, also needs to support these new algorithms. This includes certificate generation, digital signatures, certificate validation, authentication, and secure communication protocols.

During this internship, my work is focused on understanding these concepts step by step and implementing them practically. The idea is not just to learn how the algorithms work, but also to understand how they can be integrated into a real PKI system.

1.2 Internship Objective

The main objective of this internship is to understand **Post-Quantum Cryptography** and learn how it can be used to build a secure **Public Key Infrastructure (PKI)**.

Instead of directly implementing everything, I first started by studying the theory behind quantum computing, post-quantum cryptography, digital certificates, and PKI. After understanding the concepts, I gradually started implementing different components to gain practical experience.

The major objectives of this internship are:

- To understand the security challenges introduced by quantum computing.
- To study NIST standardized post-quantum cryptographic algorithms.
- To explore post-quantum algorithms using the **liboqs** library.
- To understand the working of Public Key Infrastructure (PKI).
- To study the X.509 certificate standard.
- To implement important PKI components from scratch for better understanding.

- To study TLS 1.3 and understand how secure communication is established.
- To prepare a foundation for implementing a Post-Quantum PKI that can later be integrated into IoT environments.

1.3 About This Document

This document contains the work completed during my internship **till now**. It is **not the final internship report**. Instead, it is a technical progress logbook where I am recording everything that I have studied, implemented, experimented with, and observed during the internship.

Wherever possible, I have included both the theoretical concepts and their practical implementation together. This makes it easier to understand how each concept was applied in practice.

Since the internship is still in progress, this document will continue to grow as new topics are studied and more implementations are completed.

1.4 Current Status of the Project

At the time of writing this document, I am in the **Learning and Experimental Phase** of the project.

So far, I have completed the theoretical study of several post-quantum cryptographic algorithms and Public Key Infrastructure concepts. Along with the theory, I have also started implementing different PKI components to understand how they work internally instead of relying completely on existing libraries.

The current progress of the project is shown below.

Topic	Status
Post-Quantum Cryptography Fundamentals	✅ Completed
Shor's Algorithm	✅ Completed
ML-KEM (Kyber)	✅ Theory Studied
ML-DSA (Dilithium)	✅ Theory Studied
Falcon	✅ Theory Studied & Experimented
SLH-DSA	✅ Theory Studied

Topic	Status
Hybrid Cryptography	✅ Theory Studied
Public Key Infrastructure	✅ Studied
X.509 Certificate	✅ Studied
ASN.1 & DER Encoding	✅ Implemented
Certificate Builder	✅ Implemented
Certificate Validation	✅ Implemented
Certificate Chain	✅ Implemented
Communication Workflow	✅ Implemented
Authentication	✅ Implemented
TLS 1.3	🟡 **Work in Progress**
ML-KEM Integration with PKI	⏳ Planned
PQC-enabled IoT Infrastructure	⏳ Future Work

Foundations of Post-Quantum Cryptography

2.1 Objective

Before learning the individual post-quantum algorithms, I wanted to understand why they are needed in the first place. In this chapter, I studied the limitations of classical cryptography, the basics of quantum computing, and how quantum computers can affect the security of today's cryptographic systems. This helped me understand the motivation behind Post-Quantum Cryptography and why organizations around the world are working towards replacing existing public-key algorithms.

2.2 Classical Cryptography and Its Limitations

Before studying Post-Quantum Cryptography, I first revised how classical public-key cryptography works.

Most secure communication systems used today depend on algorithms such as **RSA**, **Diffie-Hellman (DH)**, and **Elliptic Curve Cryptography (ECC)**. These algorithms are widely used in applications like HTTPS, VPNs, digital certificates, secure email, software signing, and banking systems.

The security of these algorithms depends on mathematical problems that are considered difficult for classical computers to solve. For example:

- RSA depends on the difficulty of factoring very large integers.
- Diffie-Hellman depends on the discrete logarithm problem.
- ECC also depends on the elliptic curve discrete logarithm problem.

Using today's classical computers, solving these problems would take an impractical amount of time, which makes these algorithms secure for present-day applications.

However, this security assumption changes when quantum computers become practical.

2.3 Quantum Computing

To understand why classical cryptography is at risk, I studied the basic concepts of quantum computing.

Unlike classical computers that use bits (0 or 1), quantum computers use **qubits**, which can exist in multiple states at the same time because of **superposition**. Quantum computers also make use of **entanglement**, allowing multiple qubits to work together in ways that are not possible in classical systems.

Because of these properties, quantum computers can solve certain mathematical problems much faster than classical computers.

Although large-scale quantum computers are still under development, continuous progress in this field makes it important to prepare cryptographic systems for the future.

2.4 Shor's Algorithm

One of the most important topics I studied was **Shor's Algorithm**, as it explains why current public-key cryptography is vulnerable to quantum computers.

Shor's Algorithm is a quantum algorithm that can efficiently solve integer factorization and discrete logarithm problems. Since RSA, Diffie-Hellman, and ECC are based on these mathematical problems, a sufficiently powerful quantum computer running Shor's Algorithm would be able to break these cryptographic systems.

During my study, I learned the high-level working of the algorithm:

- The integer to be factored is represented as **N = p × q**.
- The problem is converted into finding the period of a mathematical function.
- Quantum Phase Estimation (QPE) is used to determine this period.
- The period is then used to recover the prime factors using the Greatest Common Divisor (GCD).

Although the mathematical details are complex, understanding the overall workflow helped me understand why quantum computers pose such a serious threat to classical cryptography.

2.5 Fault-Tolerant Quantum Computing

While studying Shor's Algorithm, I also learned that today's quantum computers are not yet powerful enough to break modern cryptographic systems.

Current quantum devices suffer from high error rates and unstable qubits. To run large quantum algorithms successfully, these errors must be corrected using **Quantum Error Correction**, which combines many physical qubits to create a more reliable logical qubit.

From my study, I learned that:

- Current quantum computers are not sufficiently fault tolerant.
- Factoring a 2048-bit RSA key would require millions of physical qubits.

- Thousands of physical qubits may be required to create a single reliable logical qubit.
- As of the material I studied, the largest number factored using Shor's Algorithm on a quantum computer was 21 (3 × 7).

This showed me that although the threat is not immediate, it is realistic enough that organizations must start preparing their cryptographic infrastructure today.

2.6 Why Post-Quantum Cryptography?

After understanding the limitations of classical cryptography and the capabilities of quantum computers, the need for Post-Quantum Cryptography became much clearer.

Post-Quantum Cryptography focuses on developing algorithms that can run on classical computers while remaining secure against attacks from both classical and quantum computers.

One important point I came across during my study was the **"Harvest Now, Decrypt Later"** threat. An attacker can capture encrypted communication today and store it. Even if they cannot decrypt it now, they may be able to do so in the future once large-scale quantum computers become available. This is one of the main reasons organizations are already migrating towards quantum-resistant cryptography.

2.7 NIST Standardization

I also studied the efforts made by the National Institute of Standards and Technology (NIST) to standardize post-quantum cryptographic algorithms.

The algorithms that form the basis of my internship work are:

- **ML-KEM (FIPS 203)** for key establishment.
- **ML-DSA (FIPS 204)** for digital signatures.
- **SLH-DSA (FIPS 205)** as a hash-based digital signature algorithm.

Apart from these standardized algorithms, I also studied **Falcon**, which is known for its compact signatures and efficient performance, although it follows a different mathematical approach based on lattices.

These algorithms are explored in detail in the following chapters of this document.

2.8 What I Learned

By completing this chapter, I developed a clear understanding of:

- Why existing public-key cryptography is vulnerable to quantum computers.
- The basic working of quantum computers.
- The significance of Shor's Algorithm.
- The current limitations of quantum hardware.
- The motivation behind developing Post-Quantum Cryptography.
- The role of NIST in standardizing quantum-resistant algorithms.

This chapter provided the foundation required to understand the individual post-quantum algorithms that are discussed in the following chapters.

ML-KEM (Kyber)

3.1 Objective

After understanding why Post-Quantum Cryptography is needed, I started studying the first NIST standardized algorithm, **ML-KEM (Module Lattice-Based Key Encapsulation Mechanism)**. The objective was to understand how ML-KEM works, why lattice-based cryptography is considered quantum resistant, and how key establishment is performed using this algorithm. Along with the theory, I also explored the algorithm practically using the **liboqs** library.

3.2 What I Learned

ML-KEM, previously known as **CRYSTALS-Kyber**, is a lattice-based Key Encapsulation Mechanism (KEM) standardized by NIST under **FIPS 203**. Unlike RSA, which is used for both encryption and key exchange, ML-KEM is designed specifically for secure key establishment between two communicating parties.

During my study, I learned that ML-KEM is based on the **Module Learning with Errors (Module-LWE)** problem, which is currently believed to be resistant to attacks from both classical and quantum computers.

For my study and future implementation, I focused on the **ML-KEM-768** parameter set.

3.3 Working of ML-KEM

I studied the complete workflow of ML-KEM to understand how two users can securely establish a shared secret.

Key Generation

During key generation:

- Alice selects the required random values and generates the public matrix and secret vectors.
- Using these values, the public key is generated.
- The secret vector is stored as the private key.

At the end of this process:

- Public Key = (A, t)
- Private Key = s

Encapsulation (Encryption)

To establish a shared secret:

- Bob first obtains Alice's public key.
- Bob generates fresh random values.
- These values are used to compute the ciphertext.
- The ciphertext is then sent to Alice.

The ciphertext consists of two values:

- u
- v

Together they form:

Ciphertext = (u, v)

Decapsulation (Decryption)

After receiving the ciphertext, Alice uses her private key to recover the original shared secret.

This allows both Alice and Bob to independently compute the same secret key without transmitting it directly over the communication channel.

3.4 Optimizations in ML-KEM

While studying the algorithm, I also learned about some important optimizations that make ML-KEM practical for real-world applications.

Smaller Public Keys

Instead of storing the complete public matrix **A**, ML-KEM stores only a random seed. The matrix can later be regenerated whenever required.

This reduces the public key size significantly.

For example:

- Original public key size: **4608 bytes**
- Optimized public key size: **1184 bytes**

Ciphertext Compression

Another optimization used in ML-KEM is ciphertext compression.

The lower-order bits of polynomial coefficients are discarded before transmission. This reduces the ciphertext size while maintaining correctness during decryption.

Other Optimizations

During my study, I also came across:

- Central Binomial Distribution
- Fast Polynomial Multiplication

Although I did not study these mathematical concepts in detail during this phase, I understood that they are important for improving the efficiency of ML-KEM.

3.5 Practical Implementation

After understanding the theoretical concepts of ML-KEM, I started exploring its practical implementation using the **Open Quantum Safe (liboqs)** Python library. The main objective of this implementation was to understand how ML-KEM can be used in software applications and to compare different parameter sets provided by the library.

Experiment 1 – Exploring liboqs Support for KEM Algorithms

Before implementing ML-KEM, I first explored the algorithms available in the liboqs library.

The exploration showed that the installed version of liboqs supports:

Property	Value
Total Key Encapsulation Mechanisms (KEMs)	**29**
Total Signature Algorithms	**221**

Among the supported KEM algorithms, the following ML-KEM variants were available:

- ML-KEM-512
- ML-KEM-768
- ML-KEM-1024

Along with these, several other post-quantum KEMs such as Classic McEliece, FrodoKEM, HQC, BIKE and NTRU were also available for experimentation.

Observation

The library provides a common API for all supported KEM algorithms, making it easy to compare different post-quantum schemes without changing the implementation logic.

Experiment 2 – Studying ML-KEM Parameter Sets

The next experiment focused on studying the three standardized ML-KEM parameter sets.

For each variant, I generated a key pair, performed encapsulation, decapsulation, and recorded the output sizes.

Algorithm	Public Key	Ciphertext	Shared Secret
ML-KEM-512	**800 Bytes**	**768 Bytes**	**32 Bytes**
ML-KEM-768	**1184 Bytes**	**1088 Bytes**	**32 Bytes**
ML-KEM-1024	**1568 Bytes**	**1568 Bytes**	**32 Bytes**

For all three parameter sets:

- Key generation completed successfully.
- Encapsulation completed successfully.
- Decapsulation successfully recovered the shared secret.
- The sender and receiver generated identical shared secrets.

Observation

As the security level increased, the public key and ciphertext sizes also increased. However, the shared secret size remained constant at **32 bytes** for all three parameter sets.

Experiment 3 – Performance Benchmark

To compare the practical performance of different KEM algorithms, I conducted a benchmark using **1000 iterations** for each algorithm.

The benchmark measured:

- Average Key Generation Time
- Average Encapsulation Time
- Average Decapsulation Time
- Total Execution Time

The algorithms selected for comparison were:

- ML-KEM-512

- ML-KEM-768
- ML-KEM-1024
- Kyber512
- FrodoKEM-640-AES

The benchmark results are summarized below.

Algorithm	Key Generation	Encapsulation	Decapsulation	Total
ML-KEM-512	**0.289 ms**	**0.351 ms**	**0.446 ms**	**1.087 ms**
ML-KEM-768	**0.329 ms**	**0.375 ms**	**0.451 ms**	**1.155 ms**
ML-KEM-1024	**0.479 ms**	**0.530 ms**	**0.628 ms**	**1.636 ms**
Kyber512	**0.170 ms**	**0.224 ms**	**0.238 ms**	**0.632 ms**
FrodoKEM-640-AES	**6.693 ms**	**6.429 ms**	**6.392 ms**	**19.513 ms**

Observation

The benchmark clearly showed that increasing the security level results in increased computation time. ML-KEM-512 was the fastest among the ML-KEM variants, while ML-KEM-1024 required the highest execution time due to its larger parameter sizes.

The benchmark also highlighted the performance difference between lattice-based algorithms (ML-KEM/Kyber) and code-based algorithms such as FrodoKEM.

3.6 Experimental Results

The implementation was successfully completed for all three standardized ML-KEM variants.

The following results were observed during experimentation:

- Successfully explored **29** Key Encapsulation Mechanisms available in liboqs.
- Verified support for all three standardized ML-KEM variants.
- Successfully generated key pairs for ML-KEM-512, ML-KEM-768, and ML-KEM-1024.
- Successfully performed key encapsulation and decapsulation for each parameter set.
- Verified that both communicating parties generated the same shared secret in every experiment.

- Compared the public key sizes, ciphertext sizes, and shared secret sizes of different ML-KEM variants.
- Measured the execution time of different KEM algorithms and compared their performance.

Key Findings

From these experiments, the following observations were made:

- **ML-KEM-512** provided the lowest execution time among the standardized ML-KEM variants.
- **ML-KEM-1024** offered the highest security level but required larger keys, larger ciphertexts, and more computation time.
- **ML-KEM-768** provided a good balance between security and performance, making it a practical choice for future implementations.
- The shared secret generated by ML-KEM remained **32 bytes** irrespective of the parameter set.
- The liboqs library provided a consistent and easy-to-use interface for experimenting with different post-quantum algorithms.
- The benchmark results demonstrated that lattice-based KEMs are significantly more efficient than some alternative post-quantum KEMs such as FrodoKEM under the tested conditions.

ML-DSA (Dilithium)

4.1 Objective

After studying ML-KEM for secure key establishment, I moved on to the next NIST standardized algorithm, **ML-DSA (Module Lattice-Based Digital Signature Algorithm)**. The objective was to understand how post-quantum digital signatures work, how messages are authenticated, and how integrity can be verified using lattice-based cryptography. Along with studying the theory, I also implemented ML-DSA using the **liboqs** library to understand the complete signing and verification process.

4.2 What I Learned

ML-DSA, previously known as **CRYSTALS-Dilithium**, is a **Module Lattice-Based Digital Signature Algorithm** standardized by NIST under **FIPS 204**. Unlike ML-KEM, which is used for secure key establishment, ML-DSA is used to provide authentication, integrity, and non-repudiation through digital signatures.

During my study, I learned that ML-DSA uses lattice-based mathematics instead of number-theoretic problems such as integer factorization or discrete logarithms. Because of this, it is considered resistant to attacks from both classical and quantum computers.

For my study and future implementation, I mainly focused on **ML-DSA-65**, as it offers a good balance between security and computational performance.

4.3 Working of ML-DSA

I studied the complete workflow of ML-DSA to understand how secure digital signatures are generated and verified.

Key Generation

- Select $A \in_R R_q^{kxl}$, $s_1 \in_R S_\eta^l$ and $S_2 \in_R S_n^k$.
- Compute $t = As_1 + s_2$.
- Alice verification public key is (A,t) and private key is (s_1,s_2).
- The generated keys are:
    - **Verification (Public) Key = (A, t)**
    - **Signing (Private) Key = (s_1, s_2)**

Signature Generation

5 **while** Found = false **do**
6 Select $y \in_R \tilde{S}^{\ell}_{\gamma_1}$.
7 Compute $w = Ay$ and $w_1 = \text{HighBits}(w)$.
8 Compute $c = H(M \| w_1)$.
9 **if** LowBits$(w - cs_2)$ are "sufficiently small" **then**
10 Compute $z = y + cs_1$.
11 Found $\leftarrow$ true.
12 Return$(\sigma \leftarrow (c, z))$.

Signature Verification

13 Obtain an authentic copy of Alice's verification key (A, t).
14 Compute $w'_1 = \text{HighBits}(Az - ct)$.
15 Accept if and only if $c = H(M \| w'_1)$.

4.4 Features of ML-DSA

While studying ML-DSA, I came across several characteristics that make it suitable for future post-quantum systems.

Quantum Resistance

ML-DSA is based on module lattice problems, which are currently believed to remain secure against attacks from quantum computers.

Digital Authentication

The algorithm provides digital signatures that allow receivers to verify both the authenticity and integrity of a message.

NIST Standardization

ML-DSA has been standardized under **FIPS 204**, making it one of the primary digital signature algorithms recommended for future post-quantum deployments.

Parameter Sets

ML-DSA is available in three standardized parameter sets:

- ML-DSA-44
- ML-DSA-65
- ML-DSA-87

Each parameter set offers a different trade-off between security, key size, signature size, and computational performance.

4.5 Practical Implementation

After understanding the theoretical concepts of ML-DSA, I explored its practical implementation using the **Open Quantum Safe (liboqs)** Python library.

The objective of this implementation was to understand the complete signing and verification process while comparing different ML-DSA parameter sets.

Experiment 1 – Exploring ML-DSA Parameter Sets

The first experiment involved exploring the three standardized ML-DSA variants available in liboqs.

The variants studied were:

- ML-DSA-44
- ML-DSA-65
- ML-DSA-87

For each parameter set, I generated a key pair, signed a message, verified the generated signature, and recorded the execution time.

Experiment 2 – Digital Signature Generation and Verification

For each ML-DSA variant, the following steps were performed:

- Generate a public/private key pair.
- Sign a sample message.
- Verify the generated signature.
- Measure key generation, signing, and verification time.

The implementation results are summarized below.

Algorithm	Public Key	Signature Size	Verification
ML-DSA-44	**1312 Bytes**	**2420 Bytes**	Valid
ML-DSA-65	**1952 Bytes**	**3309 Bytes**	Valid
ML-DSA-87	**2592 Bytes**	**4627 Bytes**	Valid

For all three parameter sets:

- Key generation completed successfully.
- Messages were signed successfully.
- Signature verification completed successfully.
- Every generated signature was verified correctly.

Observation

As the security level increased, both the public key size and signature size increased. However, all three parameter sets successfully authenticated the message without any verification failures.

Experiment 3 – Performance Benchmark

To compare the computational performance of different ML-DSA variants, I conducted a benchmark using **1000 iterations** for each parameter set.

The benchmark measured:

- Average Key Generation Time
- Average Signing Time
- Average Verification Time
- Total Execution Time

The benchmark results are summarized below.

Algorithm	Key Generation	Signing	Verification	Total
ML-DSA-44	**0.549 ms**	**1.845 ms**	**0.592 ms**	**2.986 ms**
ML-DSA-65	**1.479 ms**	**4.566 ms**	**1.518 ms**	**7.563 ms**
ML-DSA-87	**2.964 ms**	**7.761 ms**	**3.029 ms**	**13.754 ms**

Observation

The benchmark showed that increasing the security level increases the computational cost of key generation, signing, and verification. Among the three variants, ML-DSA-44 was the fastest, while ML-DSA-87 required the highest execution time due to its larger parameter sizes.

Experiment 4 – Tamper Detection

To verify the integrity protection provided by ML-DSA, I performed a tamper detection experiment.

The experiment consisted of:

- Signing an original message.
- Modifying the message after it had been signed.
- Attempting to verify the original signature using the modified message.

The implementation produced the following results:

Test	Result
Original Message Verification	PASS
Tampered Message Verification	FAIL

Observation

The modified message failed signature verification, demonstrating that ML-DSA can successfully detect message tampering. This confirms that any change made to the signed data invalidates the signature and prevents unauthorized modifications from going unnoticed.

4.6 Experimental Results

The implementation was successfully completed for all three standardized ML-DSA variants.

The following results were observed during experimentation:

- Successfully implemented ML-DSA-44, ML-DSA-65, and ML-DSA-87.
- Successfully generated digital signature key pairs.
- Successfully signed messages using each parameter set.
- Successfully verified all generated signatures.
- Compared public key sizes and signature sizes for different security levels.
- Measured key generation, signing, and verification time for all variants.
- Successfully detected message tampering through signature verification.

Key Findings

From these experiments, the following observations were made:

- **ML-DSA-44** provided the fastest execution time among the three variants.
- **ML-DSA-87** offered the highest security level but required the largest keys, largest signatures, and highest computation time.
- **ML-DSA-65** provided a balanced trade-off between performance and security, making it a suitable choice for future implementations.
- Signature verification succeeded only when the original message remained unchanged.
- Any modification to the message immediately caused signature verification to fail.
- The liboqs library provided a consistent interface for all ML-DSA parameter sets, simplifying experimentation and comparison.

4.7 Real-World Applications

During my study, I also explored some of the practical applications of ML-DSA.

Some of the major applications include:

- Digital certificate signing in Public Key Infrastructure (PKI).
- Software and firmware code signing.
- Secure document authentication.
- Identity verification systems.
- Secure communication protocols requiring digital signatures.
- Authentication for IoT devices and embedded systems.

These applications demonstrate why ML-DSA has become one of the primary digital signature algorithms selected by NIST for post-quantum cryptography.

4.8 Observations

Studying ML-DSA helped me understand how post-quantum digital signatures differ from classical signature algorithms.

Although the underlying lattice mathematics is complex, implementing the algorithm using liboqs made it easier to understand the complete signing and verification workflow.

The tamper detection experiment clearly demonstrated the importance of digital signatures in ensuring message integrity and authenticity. It also showed that ML-DSA can effectively detect even small modifications to a signed message.

SLH-DSA (FIPS 205)

5.1 Objective

After studying lattice-based digital signature algorithms, I wanted to understand another NIST standardized signature algorithm that follows a completely different approach. The objective was to study **SLH-DSA (Stateless Hash-Based Digital Signature Algorithm)**, understand how hash-based signatures work, and compare them with lattice-based signature algorithms such as ML-DSA. Along with studying the theory, I also implemented SLH-DSA using the **liboqs** library to understand its practical working.

5.2 What I Learned

SLH-DSA is a **Stateless Hash-Based Digital Signature Algorithm** standardized by NIST under **FIPS 205**. Unlike ML-DSA, which is based on lattice cryptography, SLH-DSA derives its security entirely from cryptographic hash functions.

The main idea behind SLH-DSA is to use a **hypertree** consisting of a very large number of leaves. Each leaf represents a one-time signature (OTS) key pair. Whenever a message needs to be signed, one of these one-time keys is selected. Since every key is intended to be used only once, the probability of key reuse is kept negligibly small.

Unlike traditional hash-based signature schemes that require the signer to maintain state information, SLH-DSA is **stateless**, making it easier to deploy in practical systems without worrying about synchronization issues.

5.3 Working of SLH-DSA

I studied the complete workflow of SLH-DSA to understand how digital signatures are generated and verified using hash-based cryptography.

Key Generation

During key generation:

- Alice selects random values (x_i) for each one-time signature key.
- Each private key is processed using a preimage-resistant hash function **G** to generate the corresponding public key.
- The collection of private keys forms Alice's private key, while the hashed values form the public key.

At the end of this process:

- **Private Key = X = (x_0, x_1, x_2, ...)**
- **Public Key = Y = (y_0, y_1, y_2, ...)**

Signature Generation

To sign a message:

- Alice first generates a random message randomizer **C**.
- She computes the hash:

h = H(C, M)

- The hash is divided into multiple smaller blocks.
- Based on these hash values, the corresponding one-time private keys are selected.
- The selected keys together form the signature.

The generated signature is:

Signature = (C, S)

where **S** contains the selected one-time signature values.

Signature Verification

To verify the received signature:

- Bob first obtains Alice's authentic public key.
- He computes the hash of the received message together with the randomizer.
- Each component of the received signature is hashed again using **G**.
- The generated values are compared with Alice's public key.

If all generated values match the corresponding public key values, the signature is accepted; otherwise, it is rejected.

This process ensures both message integrity and sender authentication.

5.4 Key Characteristics of SLH-DSA

While studying SLH-DSA, I came across several characteristics that make it different from lattice-based signature algorithms.

Hash-Based Security

The security of SLH-DSA depends entirely on cryptographic hash functions rather than lattice mathematics or number-theoretic problems.

Stateless Design

Unlike earlier hash-based signature schemes, SLH-DSA does not require maintaining signing state information, making implementation simpler and reducing the risk of state synchronization errors.

Hypertree Structure

Instead of relying on a single Merkle tree, SLH-DSA uses a hypertree structure that combines multiple tree levels, allowing a very large number of signatures while keeping the probability of key reuse extremely low.

NIST Standardization

SLH-DSA has been standardized by NIST under **FIPS 205**, making it one of the officially recommended post-quantum digital signature algorithms.

5.5 Practical Implementation

After understanding the theoretical concepts of SLH-DSA, I explored its practical implementation using the **Open Quantum Safe (liboqs)** Python library. The objective of this implementation was to understand the complete signing and verification workflow of a hash-based digital signature algorithm and to evaluate its behaviour under different verification scenarios.

Experiment 1 – Key Pair Generation

The first experiment involved generating an SLH-DSA key pair using the **SPHINCS+-SHA2-256f-simple** parameter set.

After successfully generating the key pair, the verification key and signing key were obtained. The generated key sizes are shown below.

PARAMETER	VALUE
ALGORITHM	SPHINCS+-SHA2-256f-simple
VERIFICATION KEY SIZE	**64 Bytes**
SIGNING KEY SIZE	**128 Bytes**

The successful generation of both keys confirmed that the selected SLH-DSA parameter set was correctly initialized and ready for digital signature operations.

Observation

The generated key sizes were relatively small compared to the signature size, indicating that SLH-DSA primarily trades larger signatures for strong quantum-resistant security.

Experiment 2 – Digital Signature Generation

The next experiment focused on generating a digital signature for a sample message.

Before signing the message, its SHA-256 hash was computed. The signing key was then used to generate the digital signature.

The generated signature had the following characteristics.

PARAMETER	VALUE
SIGNATURE SIZE	**49,856 Bytes**

The signature was generated successfully without any errors.

Observation

Unlike lattice-based signature schemes, SLH-DSA produces significantly larger signatures because it relies on a hypertree structure and one-time signature schemes to achieve quantum resistance.

Experiment 3 – Signature Verification

After generating the signature, the verification key was used to verify the signed message.

The verification process completed successfully and confirmed that the generated signature matched the original message.

VERIFICATION TEST	RESULT
ORIGINAL SIGNATURE VERIFICATION	✅ Valid

Observation

The successful verification confirmed that the generated signature was authentic and that the message had not been modified after signing.

Experiment 4 – Tamper Detection and Security Validation

To evaluate the integrity protection provided by SLH-DSA, several additional verification tests were performed.

The following scenarios were tested:

- Verification after modifying the original message.
- Verification using a forged signature.
- Verification using an incorrect verification key.

The results obtained are summarized below.

TEST	RESULT
ORIGINAL MESSAGE VERIFICATION	✅ Accepted
TAMPERED MESSAGE	❌ Rejected
FORGED SIGNATURE	❌ Rejected
WRONG VERIFICATION KEY	❌ Rejected

Observation

The implementation correctly rejected every invalid verification attempt. Even a small modification to the original message caused signature verification to fail, demonstrating that SLH-DSA effectively protects both message integrity and authenticity.

Experiment 5 – Performance Benchmark

To compare different SLH-DSA parameter sets, a benchmark was conducted using **20 iterations** for each algorithm.

The benchmark measured:

- Average Key Generation Time
- Average Signing Time
- Average Verification Time

- Total Execution Time

The following parameter sets were evaluated:

- SPHINCS+-SHA2-128f-simple
- SPHINCS+-SHA2-192f-simple
- SPHINCS+-SHA2-256f-simple
- SPHINCS+-SHA2-128s-simple
- SPHINCS+-SHA2-256s-simple

The benchmark results are summarized below.

Algorithm	Public Key	Signature Size	Key Generation	Signing	Verification	Total
SPHINCS+-SHA2-128f-simple	32 Bytes	17,088 Bytes	7.488 ms	169.010 ms	10.059 ms	186.557 ms
SPHINCS+-SHA2-192f-simple	48 Bytes	35,664 Bytes	17.531 ms	435.517 ms	24.321 ms	477.369 ms
SPHINCS+-SHA2-256f-simple	64 Bytes	49,856 Bytes	43.567 ms	977.700 ms	38.675 ms	1059.942 ms
SPHINCS+-SHA2-128s-simple	32 Bytes	7,856 Bytes	479.536 ms	4301.839 ms	42.109 ms	4823.485 ms
SPHINCS+-SHA2-256s-simple	64 Bytes	29,792 Bytes	896.953 ms	9552.631 ms	49.320 ms	10498.903 ms

Observation

The benchmark highlighted the performance differences between the **Fast (f)** and **Small (s)** parameter sets. The Fast variants completed signing operations much more quickly, while the small variants significantly reduced signature size at the cost of much higher computational time.

5.6 Experimental Results

The implementation of **SLH-DSA (FIPS 205)** was successfully completed using the **SPHINCS+-SHA2-256f-simple** parameter set. The complete digital signature workflow, including key generation, message signing, verification, integrity validation, and performance benchmarking, was successfully implemented.

The following results were obtained during experimentation:

- Successfully generated an SLH-DSA key pair.
- Generated a **64-byte verification key** and a **128-byte signing key**.
- Successfully generated a digital signature of **49,856 bytes**.
- Successfully verified the generated signature using the corresponding verification key.
- Successfully detected modifications made to the original message.
- Successfully rejected forged signatures.
- Successfully rejected verification attempts using an incorrect verification key.
- Benchmarked five different SLH-DSA parameter sets and compared their performance.

Key Findings

From these experiments, the following observations were made:

- The **SPHINCS+-SHA2-256f-simple** implementation successfully completed the complete signing and verification workflow.
- Signature verification succeeded only when the original message and the correct verification key were used.
- Any modification to the message immediately caused verification to fail, demonstrating strong integrity protection.
- Verification also failed for forged signatures and incorrect verification keys, confirming the robustness of the verification process.
- Compared to ML-DSA, SLH-DSA produces considerably larger signatures because of its hash-based hypertree design.
- The **Fast (f)** variants achieved significantly lower execution times than the **Small (s)** variants.
- The **Small (s)** variants reduced signature size but required substantially higher key generation and signing times.

- Among the tested algorithms, **SPHINCS+-SHA2-128f-simple** achieved the best overall execution time (**186.557 ms**), while **SPHINCS+-SHA2-256s-simple** required the highest computational time (**10,498.903 ms**).
- The benchmark clearly demonstrated the trade-off between **security level, signature size, and computational performance**, allowing different parameter sets to be selected based on application requirements.
- The experiments confirmed that the **liboqs** implementation provides a reliable and consistent platform for experimenting with SLH-DSA and understanding the practical behaviour of hash-based digital signatures.

5.7 Real-World Applications

During my study, I also explored some practical applications of SLH-DSA.

Some of the major applications include:

- TLS and QUIC secure communication.
- Cloud security platforms.
- Secure Boot and Firmware Authentication.
- Automotive and Industrial IoT systems.
- Aerospace and satellite firmware authentication.
- Hardware Security Modules (HSMs).

These applications demonstrate that SLH-DSA is particularly suitable for environments requiring long-term security and highly reliable firmware authentication.

5.8 Observations

Studying SLH-DSA helped me understand that post-quantum digital signatures are not limited to lattice-based cryptography. Hash-based signatures provide an entirely different approach to achieving quantum resistance.

Although SLH-DSA produces larger signatures than ML-DSA, it offers strong security based solely on well-established cryptographic hash functions. Implementing the algorithm using liboqs helped me understand the complete signing and verification process while also highlighting the practical trade-offs between different post-quantum signature schemes.

Falcon

6.1 Objective

After studying the NIST standardized digital signature algorithms ML-DSA and SLH-DSA, I explored **Falcon (Fast-Fourier Lattice-based Compact Signatures over NTRU)** to understand another lattice-based post-quantum signature scheme. The objective was to study how Falcon differs from ML-DSA, understand its compact signature design, and evaluate its practical implementation using the **liboqs** library.

6.2 What I Learned

Falcon is a lattice-based digital signature algorithm based on the **NTRU lattice problem** together with the **GPV (Gentry, Peikert and Vaikuntanathan)** signature framework. Unlike ML-DSA, which focuses on balancing performance and implementation simplicity, Falcon is designed to produce **very compact digital signatures** while maintaining a high level of security.

One of the major advantages of Falcon is its small signature size, making it suitable for environments where communication bandwidth and storage are limited.

During my study, I also learned that Falcon uses advanced mathematical techniques such as **Fast Fourier Sampling (FFT)** and **Gaussian Sampling**, making the implementation more complex than ML-DSA.

For my practical implementation, I focused on the **Falcon-1024** parameter set.

6.3 Working of Falcon

I studied the complete workflow of Falcon to understand how compact lattice-based digital signatures are generated and verified.

Key Generation

NTRU lattice gives polynomial ring:

$$R_q = \mathbb{Z}_q[x]/(x^n+1),\ q=12289,\ n \in \{512,1024\}$$

The private key is 4 short polynomials - $f,g,F,G \in R$ satisfying the NTRU equation:

$$fG-gF = q \pmod{\Phi},\ \Phi(x) = x^n+1$$

From f and g, public key h is computed as

$h = g \cdot f^{-1} \bmod q$

The private key matrix (secret short basis) is:

$$B_{\text{secret}} = \begin{bmatrix} g & -f \\ G & -F \end{bmatrix}$$

The public key matrix (bad/long basis) is:

$$B_{\text{public}} = \begin{bmatrix} -h/q & I_n \\ qI_n & O_n \end{bmatrix}$$

Signature Generation

- Hash message m with nonce r into polynomial: **$c = H(r||m) \in R_q$**.
- Use secret basis to find two short polynomials s1, s2 satisfying **$s1 + s2.h = c \bmod q$.**
- The signature is s2 since s1 can be recomputed.

Gaussian Sampling

Falcon samples s2 from discreet Gaussian distribution centred near the target lattice point.

This requires:

- LDL decomposition of the Gram-Schmidt basis.
- Recursive FFT based sampling using a structure called the Falcon Tree.
- The sampler draws from $D_\sigma(\mathbf{x}) = S^{-1} \exp\left(\frac{-\|\mathbf{x}-\mu\|^2}{2\sigma^2}\right)$

This ensures the signature statistically hides the secret basis — no information leaks about f,g,F,G.

Signature Verification

Given s2(signature) and c(hash of message):

- Recover s1 = c-s2.h mod q
- Accept if Euclidean norm satisfies.
- $\|(s_1,s_2)\|^2 \leq \lfloor \beta^2 \rfloor$
- Falcon-512: $\lfloor \beta^2 \rfloor = 34,034,726$
- Falcon-1024: $\lfloor \beta^2 \rfloor = 70,265,242$

6.4 Key Characteristics of Falcon

While studying Falcon, I came across several characteristics that distinguish it from other post-quantum signature schemes.

Compact Signatures

Falcon produces significantly smaller signatures compared to many other post-quantum signature algorithms, making it attractive for communication systems with limited bandwidth.

Lattice-Based Security

Falcon derives its security from the hardness of lattice problems, which are believed to remain secure against attacks from both classical and quantum computers.

FFT-Based Sampling

Falcon uses Fast Fourier Transform (FFT) techniques together with Gaussian sampling to generate efficient and compact signatures.

High Computational Efficiency

Although key generation is relatively expensive, signature generation and verification are efficient once the key pair has been created.

6.5 Practical Implementation

After understanding the theoretical concepts of Falcon, I explored its practical implementation using the **Open Quantum Safe (liboqs)** Python library. The objective of this implementation was to understand the complete signing and verification workflow

while evaluating the key sizes, signature sizes, verification behaviour, and performance of Falcon.

Experiment 1 – Falcon Key Pair Generation

The first experiment involved generating a Falcon key pair using the **Falcon-1024** parameter set.

After successfully generating the key pair, the verification key and signing key were obtained.

PARAMETER	VALUE
ALGORITHM	Falcon-1024
VERIFICATION KEY SIZE	**1793 Bytes**
SIGNING KEY SIZE	**2305 Bytes**

The successful generation of both keys confirmed that the Falcon implementation was correctly initialized and ready for digital signature operations.

Observation

Compared to ML-DSA and SLH-DSA, Falcon generates relatively compact verification keys while maintaining strong security.

Experiment 2 – Digital Signature Generation

A sample message was digitally signed using the generated Falcon signing key.

The generated signature characteristics are shown below.

PARAMETER	VALUE
SIGNATURE SIZE	**1270 Bytes**

The signature was successfully generated without any errors.

Observation

The generated Falcon signature was significantly smaller than the signature produced by SLH-DSA, demonstrating Falcon's focus on compact digital signatures.

Experiment 3 – Signature Verification

The generated signature was verified using the corresponding verification key.

VERIFICATION TEST	RESULT
ORIGINAL SIGNATURE VERIFICATION	✅ Valid

The verification completed successfully, confirming that the generated signature matched the original message.

Observation

The successful verification confirmed both the authenticity of the signer and the integrity of the transmitted message.

Experiment 4 – Tamper Detection and Security Validation

To evaluate the integrity protection provided by Falcon, several additional verification tests were performed.

The following scenarios were evaluated:

- Verification after modifying the original message.
- Verification using a forged signature.
- Verification using an incorrect verification key.

The results obtained are summarized below.

TEST	RESULT
ORIGINAL MESSAGE VERIFICATION	✅ Accepted
TAMPERED MESSAGE	❌ Rejected
FORGED SIGNATURE	❌ Rejected
WRONG VERIFICATION KEY	❌ Rejected

Observation

Falcon successfully rejected every invalid verification attempt. Any modification to the message, signature, or verification key immediately caused signature verification to fail, demonstrating strong protection against tampering and forgery.

Experiment 5 – Performance Benchmark

To compare Falcon parameter sets, a benchmark was conducted using **1000 iterations** for each algorithm.

The benchmark measured:

- Average Key Generation Time
- Average Signing Time
- Average Verification Time
- Total Execution Time

The results obtained are summarized below.

Algorithm	Public Key	Signature Size	Key Generation	Signing	Verification	Total
Falcon-512	897 Bytes	657 Bytes	29.545 ms	2.223 ms	0.230 ms	31.998 ms
Falcon-1024	1793 Bytes	1272 Bytes	100.686 ms	6.451 ms	0.653 ms	107.791 ms

Observation

Increasing the Falcon security level resulted in larger public keys, larger signatures, and higher execution times. Falcon-512 provided better performance, while Falcon-1024 offered a higher security level with additional computational cost.

6.6 Experimental Results

The implementation of **Falcon** was successfully completed using the **Falcon-1024** parameter set. The complete digital signature workflow, including key generation, message signing, verification, tamper detection, and performance benchmarking, was successfully implemented.

The following results were obtained during experimentation:

- Successfully generated Falcon key pairs.
- Generated a **1793-byte verification key** and a **2305-byte signing key**.
- Successfully generated a digital signature of **1270 bytes**.
- Successfully verified the generated signature using the corresponding verification key.
- Successfully detected tampered messages.
- Successfully rejected forged signatures.
- Successfully rejected verification attempts using an incorrect verification key.

- Benchmarked both Falcon parameter sets and compared their computational performance.

Key Findings

From these experiments, the following observations were made:

- Falcon successfully completed the complete digital signature workflow, including key generation, signing, verification, and integrity validation.
- Signature verification succeeded only for the original message and failed whenever the message, signature, or verification key was modified.
- Falcon produced significantly smaller signatures than SLH-DSA, making it well suited for bandwidth-constrained communication systems.
- Falcon-512 achieved the best overall performance with a total execution time of **31.998 ms**, while Falcon-1024 required **107.791 ms** because of its higher security level.
- Key generation accounted for the largest portion of the execution time, whereas signature verification remained comparatively fast for both parameter sets.
- The benchmark demonstrated the trade-off between security level, key size, signature size, and computational performance.
- The liboqs implementation provided a reliable platform for understanding and evaluating Falcon's practical behaviour.

6.7 Real-World Applications

During my study, I also explored some of the practical applications of Falcon.

Some of the major applications include:

- Digital certificate signing in Public Key Infrastructure (PKI).
- TLS and secure communication protocols.
- Secure firmware and software code signing.
- Embedded and IoT systems where compact signatures are beneficial.
- Hardware Security Modules (HSMs).
- Secure document authentication.

These applications highlight Falcon's suitability for environments where bandwidth, storage, and communication efficiency are important considerations.

6.8 Observations

Studying Falcon helped me understand that different lattice-based signature schemes are designed with different priorities. While ML-DSA focuses on balanced performance and simpler implementation, Falcon emphasizes compact signatures by using more advanced mathematical techniques such as FFT and Gaussian sampling.

The practical implementation using liboqs helped me understand the complete signing and verification workflow while also demonstrating Falcon's ability to detect message tampering, forged signatures, and incorrect verification keys.

Hybrid Cryptography System

7.1 Objective

After studying the individual post-quantum algorithms, I wanted to understand how they can work together in a real communication system. The objective of this implementation was to build a **Hybrid Cryptography System** by combining **ML-KEM-768**, **AES-256-GCM**, and **ML-DSA-65** to achieve secure key establishment, confidential message encryption, digital authentication, and integrity verification.

This implementation demonstrates how multiple cryptographic primitives can be integrated to provide end-to-end secure communication between two users.

7.2 What I Learned

While studying hybrid cryptography, I learned that no single cryptographic algorithm can provide all the required security services.

Different algorithms are designed for different purposes:

- **ML-KEM** is used for secure key establishment.
- **AES-256-GCM** provides efficient symmetric encryption for protecting message confidentiality.
- **ML-DSA** provides digital signatures to ensure authentication, integrity, and non-repudiation.

By combining these algorithms, a complete secure communication system can be built that leverages the strengths of each algorithm while remaining resistant to quantum attacks.

7.3 Working of the Hybrid Cryptography System

The implemented communication workflow consists of the following steps.

Phase 1 – System Setup

Before communication begins:

- Alice generates an ML-KEM key pair.
- Bob generates an ML-DSA key pair.
- Alice shares her ML-KEM public key with Bob.
- Bob shares his ML-DSA verification key with Alice.

After this phase:

- Alice is ready to receive encrypted messages.
- Bob is ready to digitally sign messages.

Phase 2 – Encryption and Digital Signature

To send a secure message:

- Bob generates a shared secret using Alice's ML-KEM public key.
- The shared secret is used as the AES-256 encryption key.
- The original message is encrypted using AES-256-GCM.
- Bob signs the encrypted package using his ML-DSA signing key.

The transmitted package contains:

- Ciphertext
- Nonce
- Authentication Tag
- ML-DSA Signature
- ML-KEM Ciphertext

Phase 3 – Verification and Decryption

After receiving the encrypted package:

- Alice first verifies Bob's ML-DSA signature.
- Alice then decapsulates the ML-KEM ciphertext to recover the shared secret.
- The recovered shared secret is used to decrypt the AES ciphertext.
- Finally, Alice obtains the original plaintext message.

Phase 4 – Tamper Detection

To evaluate the security of the communication system, the encrypted package was intentionally modified.

The modified package failed signature verification, demonstrating that unauthorized modifications are detected before decryption.

7.4 Key Characteristics

During this implementation, I observed several important characteristics of a hybrid cryptographic system.

Secure Key Establishment

ML-KEM securely establishes a shared secret without transmitting the secret directly over the communication channel.

Efficient Data Encryption

AES-256-GCM encrypts the message efficiently while also providing authenticated encryption.

Digital Authentication

ML-DSA ensures that only the legitimate sender can generate a valid digital signature.

Integrity Protection

Any modification to the transmitted package is detected during signature verification before the message is decrypted.

Quantum Resistance

The combination of ML-KEM and ML-DSA provides protection against attacks from both classical and future quantum computers, while AES-256-GCM continues to provide efficient symmetric encryption.

7.5 Practical Implementation

After understanding the role of each cryptographic algorithm individually, I implemented a complete hybrid cryptographic workflow using the **Open Quantum Safe (liboqs)** library together with the **cryptography** package for AES-256-GCM encryption.

The implementation was divided into five phases.

Phase 1 – System Setup

The first phase involved generating the cryptographic keys required by Alice and Bob.

The following keys were generated:

- Alice's ML-KEM-768 key pair.
- Bob's ML-DSA-65 key pair.

The generated public keys were exchanged to establish trust between the communicating parties.

Observation

The setup phase completed successfully, and both parties were ready for secure communication.

Phase 2 – Bob Encrypts and Signs the Message

Bob prepared the following plaintext message:

"Hello Alice! This is Quantum Safe secret message from Bob!"

The following operations were then performed:

- Generated a shared secret using ML-KEM encapsulation.
- Generated an AES-256 encryption key from the shared secret.
- Encrypted the plaintext using AES-256-GCM.
- Signed the encrypted package using the ML-DSA signing key.

The generated communication package contained:

- ML-KEM Ciphertext
- AES Ciphertext
- AES Nonce
- Authentication Tag
- ML-DSA Signature

Observation

The encrypted package was generated successfully, and all cryptographic operations completed without any errors.

Phase 3 – Alice Verifies and Decrypts

After receiving the encrypted package, Alice performed the following operations:

- Verified Bob's ML-DSA signature.
- Decapsulated the ML-KEM ciphertext.
- Recovered the shared secret.

- Verified that both shared secrets matched.
- Decrypted the AES ciphertext.
- Successfully recovered the original plaintext message.

Observation

The recovered shared secret matched the sender's shared secret, and the decrypted message was identical to the original plaintext.

Phase 4 – Tamper Detection Test

To evaluate the security of the communication system, the transmitted package was intentionally modified before verification.

The modified package failed signature verification.

Test	Result
Original Package	✅ Accepted
Tampered Package	❌ Rejected

Observation

The system successfully detected message tampering before decryption, preventing the modified message from being accepted.

Phase 5 – Summary Report

Finally, a summary of the complete communication workflow was generated.

The implementation confirmed:

- Successful ML-KEM key establishment.
- Successful AES-256-GCM encryption.
- Successful ML-DSA signature generation.
- Successful signature verification.
- Successful AES decryption.
- Successful tamper detection.

7.6 Experimental Results

The hybrid cryptography system was successfully implemented by combining **ML-KEM-768**, **AES-256-GCM**, and **ML-DSA-65**.

The implementation successfully demonstrated secure communication between Alice and Bob using post-quantum cryptographic primitives together with authenticated symmetric encryption.

The following results were obtained during experimentation:

- Successfully generated ML-KEM and ML-DSA key pairs.
- Successfully established a shared secret using ML-KEM encapsulation and decapsulation.
- Successfully encrypted the plaintext using AES-256-GCM.
- Successfully generated a digital signature using ML-DSA.
- Successfully verified the generated signature before decryption.
- Successfully decrypted the ciphertext and recovered the original message.
- Successfully detected tampering before message decryption.
- Generated a complete summary report of the communication workflow.

Key Findings

From this implementation, the following observations were made:

- ML-KEM successfully established identical shared secrets for both communicating parties without transmitting the secret directly.
- AES-256-GCM provided efficient authenticated encryption for securing the message.
- ML-DSA successfully authenticated the sender and ensured message integrity.
- Signature verification was performed before decryption, preventing unauthorized or modified messages from being processed.
- Any modification made to the transmitted package caused signature verification to fail immediately.
- The combination of ML-KEM, AES-256-GCM, and ML-DSA provided confidentiality, authentication, integrity, and quantum-resistant key establishment within a single communication workflow.
- The implementation demonstrated how multiple cryptographic algorithms can be integrated into a practical end-to-end secure communication system.

7.7 Real-World Applications

During this implementation, I also explored where hybrid cryptographic systems can be applied in practice.

Some of the major applications include:

- Quantum-safe secure messaging systems.
- Post-quantum TLS implementations.
- Public Key Infrastructure (PKI).
- Secure cloud communication.
- Government and defence communication systems.
- Financial transaction systems.
- Industrial IoT and smart infrastructure.
- Secure firmware distribution and software updates.

These applications demonstrate how hybrid cryptographic architectures can provide complete security solutions by combining key establishment, encryption, and digital signatures.

7.8 Observations

Implementing the hybrid cryptography system helped me understand how different cryptographic algorithms complement each other in a practical communication environment.

Rather than studying each algorithm independently, this implementation demonstrated how ML-KEM, AES-256-GCM, and ML-DSA can work together to provide confidentiality, authentication, integrity, and secure key establishment in a single workflow.

The phase-wise implementation also gave me a much better understanding of the complete communication process followed in secure systems and served as a strong foundation for the next stage of my internship, where these concepts will be integrated into a Post-Quantum Public Key Infrastructure (PQC-PKI).

Post-Quantum Public Key Infrastructure (PQC-PKI)

8.1 Objective

After studying the individual post-quantum cryptographic algorithms and implementing a hybrid cryptography system, I wanted to understand how these algorithms can be integrated into a complete security infrastructure. The objective of this phase was to study **Public Key Infrastructure (PKI)**, understand the role of digital certificates, Certificate Authorities (CAs), trust relationships, and certificate validation, and then build a **Post-Quantum PKI** using the post-quantum algorithms studied earlier.

The implementation focuses on replacing classical cryptographic algorithms traditionally used in PKI with post-quantum alternatives while keeping the overall PKI architecture and certificate lifecycle similar to a real-world deployment.

8.2 What I Learned

Before starting the implementation, I first studied the basic concepts of Public Key Infrastructure to understand how trust is established between communicating entities.

I learned that PKI is not a single algorithm. Instead, it is a complete framework that combines cryptographic algorithms, certificate management, policies, and trust relationships to enable secure communication over untrusted networks.

A properly designed PKI provides several important security services:

- Authentication
- Confidentiality
- Integrity
- Non-repudiation
- Trust establishment

Rather than distributing public keys directly, PKI allows entities to exchange **digital certificates** that bind an entity's identity to its public key.

8.3 Key Management

One of the first topics I studied was **Key Management**.

Key Management refers to the collection of techniques and procedures used to create, distribute, store, update, and protect cryptographic keys throughout their lifecycle. Proper key management is essential because even the strongest cryptographic algorithms become ineffective if their keys are compromised.

During my study, I learned that key management includes activities such as:

- Key generation
- Secure key distribution
- Secure key storage
- Key renewal
- Key backup
- Key revocation
- Key destruction

The security of the entire PKI depends heavily on how these cryptographic keys are managed.

8.4 Methods of Public Key Distribution

Before the introduction of PKI, distributing authentic public keys was itself a major challenge.

I studied several methods through which public keys can be distributed.

These include:

- Peer-to-peer delivery over a trusted channel
- Trusted courier
- One-time user registration
- Visual inspection
- Direct access to a trusted file
- Online trusted server
- Offline Certification Authority (CA)

Although these methods can be used in different situations, they are difficult to scale in large communication systems. This is one of the primary reasons why Public Key Infrastructure was developed.

Instead of distributing raw public keys, PKI distributes **digital certificates**, allowing users to verify public keys through a trusted Certificate Authority.

8.5 Public Key Certificates

I learned that a **Public Key Certificate** is a digital document generated and digitally signed by a Certificate Authority.

Its purpose is to securely bind an entity's identity with its public key so that other users can trust the authenticity of that public key.

A certificate typically contains:

- Identity of the certificate owner
- Public key
- Validity period
- Certificate serial number
- Issuer information
- Digital signature of the Certificate Authority

When another user receives the certificate, they verify the CA's digital signature before trusting the contained public key.

This eliminates the need to exchange public keys manually while reducing the possibility of impersonation attacks.

8.6 Public Key Infrastructure (PKI)

After understanding certificates, I studied the complete Public Key Infrastructure.

PKI is a comprehensive framework that combines cryptographic protocols, technologies, policies, and supporting services to ensure the secure and reliable use of digital certificates. Rather than focusing only on encryption or digital signatures, PKI manages the complete lifecycle of certificates and establishes trust between communicating entities.

The major components of a PKI include:

- Certificate Format
- Certificate Authority
- Certification Process
- Certificate Revocation
- Trust Model
- Certificate Policy

- Certificate Practice Statement

Together, these components form the foundation of secure digital communication systems.

8.7 Components of a PKI

During my study, I explored the major building blocks of a Public Key Infrastructure.

Certificate Authority (CA)

A Certificate Authority is responsible for issuing digital certificates that bind an entity's identity to its public key.

The CA digitally signs each certificate, allowing other users to verify its authenticity using the CA's public key. Since every issued certificate depends on the CA's signature, the security of the CA is one of the most important aspects of the entire PKI.

Certificate Format

A certificate follows a predefined format so that every system can interpret and verify it consistently.

The certificate contains both the certificate data and the digital signature generated by the issuing Certificate Authority.

Trust Model

The Trust Model defines how trust is established between different entities.

Instead of trusting every user individually, entities trust the Certificate Authority. Once the CA's public key is trusted, any certificate signed by that CA can also be trusted after successful verification.

Certificate Policy

A Certificate Policy defines the intended purpose, usage, and scope of certificates issued by a particular Certificate Authority.

It specifies how certificates should be used within the PKI.

Certificate Practice Statement (CPS)

A Certificate Practice Statement describes the operational procedures followed by a Certificate Authority while issuing, managing, and revoking certificates.

It ensures that certificate management follows a consistent set of practices.

8.8 X.509 Certificates

The X.509 certificate format is the most widely used standard for digital certificates in modern Public Key Infrastructure.

I studied how an X.509 certificate is structured and how it enables trust between communicating entities.

According to the X.509 model, a Certificate Authority issues a certificate that binds an entity's identity with its public key. The certificate contains two major parts:

Data Part

The data section contains:

- Entity identity
- Public key
- Validity period
- Additional certificate information

Signature Part

The second part contains the digital signature generated by the issuing Certificate Authority.

When another user receives the certificate, they first obtain an authentic copy of the CA's public key. The CA's signature is then verified before trusting the public key contained inside the certificate.

One important point I learned is that the Certificate Authority **never needs access to the user's private key**. However, the CA must be trusted not to issue fraudulent certificates.

8.9 Certification Process

I also studied the complete lifecycle followed by a Certificate Authority while issuing a digital certificate.

The certification process consists of several stages.

CA Key Generation

The Certificate Authority first generates its own public and private key pair.

The CA's private key is considered extremely sensitive. Ideally, it should be generated and stored inside a **Hardware Security Module (HSM)** so that it never leaves the secure hardware. Multiple backup copies may also be stored in geographically separated HSMs. In some deployments, threshold signature schemes are also used for additional protection.

Certificate Request

The entity requesting a certificate submits a certificate request containing its public key.

This request should itself be authenticated, and the CA may maintain a permanent record of every certificate request received.

Identity Verification

Before issuing a certificate, the identity of the requesting entity must be verified.

This responsibility may be performed directly by the Certificate Authority or delegated to a **Registration Authority (RA)** or an intermediate Certificate Authority. After successful verification, the request is forwarded for certificate issuance.

Key Validation

The Certificate Authority verifies that the submitted public key is valid.

The requesting entity must also prove possession of the corresponding private key before the certificate is issued.

Certificate Issuance

After successful verification, the Certificate Authority generates and signs the certificate.

The certificate is then issued to the requesting entity, and the CA may require confirmation that the certificate has been received and accepted.

8.10 Certificate Revocation

Certificates are normally valid only for a fixed period. However, there are situations where a certificate must be revoked before its expiration date.

Some common reasons for certificate revocation include:

- Private key compromise
- Owner leaving the organization
- Change in the owner's role or responsibilities

Before relying on any certificate, users should verify that it has not been revoked.

I studied three common methods used to distribute revocation information:

- Short-lived certificates
- Online certificate status checking
- Certificate Revocation Lists (CRLs)

Among these, CRLs are one of the most widely used approaches.

A **Certificate Revocation List (CRL)** is a digitally signed list of revoked certificates published periodically by the Certificate Authority. Before trusting a certificate, a relying party checks that the certificate's serial number does not appear in the latest CRL.

While studying CRLs, I also learned about some practical challenges associated with their management:

- Delay between certificate revocation and CRL publication
- Large CRL size in large-scale deployments
- Increased management complexity as the PKI grows

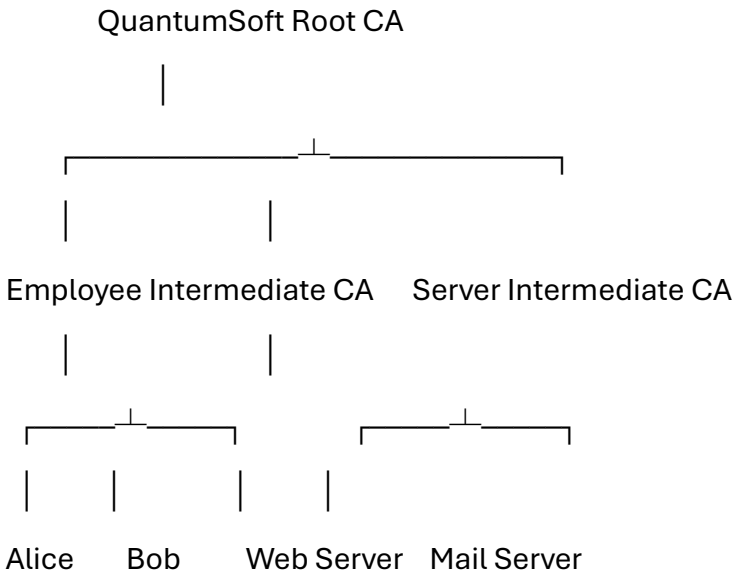
8.11 Designing the PQC-PKI Architecture

Before implementing the PKI, I first designed a small enterprise environment that closely resembles a real-world organization.

For this implementation, I assumed a software company named **QuantumSoft Pvt. Ltd.** The organization was designed with a hierarchical Certificate Authority structure

consisting of one Root Certificate Authority, two Intermediate Certificate Authorities, employee certificates, and server certificates.

The overall hierarchy of the designed PKI is shown below.



This hierarchy follows the same trust model used in most modern enterprise Public Key Infrastructures, where every certificate ultimately derives its trust from a single trusted Root Certificate Authority.

8.12 Real-World Scenario

To better understand how PKI operates in practice, I designed the implementation around a realistic enterprise scenario.

In the simulated organization:

- Employees such as **Alice**, **Bob**, and **Charlie** receive certificates from the **Employee Intermediate CA**.
- Internal services such as the **Web Server**, **Mail Server**, and future application servers receive certificates from the **Server Intermediate CA**.
- Both intermediate CAs are trusted because they are issued and signed by the **QuantumSoft Root CA**.

For example, when Alice connects to the company's web application, the certificate validation process follows the chain:

Web Server Certificate

Server Intermediate CA

QuantumSoft Root CA

Trusted

This workflow closely resembles how HTTPS certificate validation works on the Internet.

Designing the implementation around a realistic organization helped me understand how certificate hierarchies and trust relationships operate in practical deployments.

8.13 Organizational Structure

Before implementing any cryptographic operations, I defined the structure of the organization that would be managed by the PKI.

The organizational information includes:

- Organization Name
- Country
- State
- Locality
- Organizational Unit
- Root Certificate Authority
- Intermediate Certificate Authorities
- Employees
- Servers

This information is later embedded inside the X.509 certificates during certificate generation.

By defining the organization first, every certificate generated within the PKI follows a consistent identity structure.

8.14 Cryptographic Policy

The next step was to define the cryptographic policy that would be used throughout the PKI.

Rather than relying on classical cryptographic algorithms, the implementation was designed around post-quantum algorithms.

The selected algorithms are summarized below.

PURPOSE	ALGORITHM
DIGITAL SIGNATURES	ML-DSA (Primary)
EXPERIMENTAL SIGNATURES	Falcon-1024
BACKUP SIGNATURE SCHEME	ML-DSA
KEY ESTABLISHMENT *(FUTURE INTEGRATION)*	ML-KEM-768

The notebook also mentions that Falcon is treated as an **experimental algorithm**, while ML-DSA serves as the primary digital signature algorithm with backup support wherever Falcon is evaluated.

Establishing this policy before implementation ensured that all entities within the PKI followed the same cryptographic standards.

8.15 Designing the X.509 Certificate Framework

Once the organizational structure and cryptographic policy were finalized, I began designing the X.509 certificate framework.

Instead of relying entirely on an existing certificate library, the objective was to understand how certificates are actually constructed internally.

For this reason, I planned to implement the certificate structure step by step.

The certificate framework includes:

- ASN.1 data representation.
- DER encoding.
- Object Identifier (OID) registry.
- Algorithm Identifier registry.
- Subject Public Key Information.

- TBSCertificate construction.
- Certificate extensions.
- Digital signature generation.

This modular design made it easier to understand every component involved in generating an X.509 certificate.

8.16 Certificate Issuance Workflow

Before implementing certificate issuance, I first studied the complete workflow followed when a new user joins an organization.

The implemented workflow is illustrated below.

Alice joins QuantumSoft Pvt. Ltd.

Alice generates her ML-DSA key pair

Private Key remains with Alice

Public Key

Certificate Signing Request (CSR)

Root CA verifies Alice's identity

Root CA signs Alice's public key

Alice receives her X.509 certificate

This workflow closely follows the certificate issuance process used in real-world PKI deployments.

8.17 Understanding Certificate Signing Requests (CSR)

Before implementing certificate issuance, I also studied the purpose of a **Certificate Signing Request (CSR).**

A CSR is a request generated by an entity whenever it wants a Certificate Authority to issue a digital certificate.

Conceptually, the request can be understood as:

"Hello QuantumSoft Root CA, my name is Alice. Here is my public key. Please issue a certificate for me."

Although the message appears simple, the CSR is itself digitally signed by the requesting entity.

Cryptographically, the CSR contains:

- Subject information.
- Public key.
- Requested certificate extensions.
- Digital signature generated using the applicant's private key.

This digital signature proves ownership of the corresponding private key before the Certificate Authority issues the certificate.

8.18 Design Observations

Designing the PKI architecture before implementation helped me understand that a Public Key Infrastructure is much more than certificate generation.

It is a complete trust framework that defines:

- Organizational hierarchy.
- Trust relationships.
- Certificate issuance procedures.
- Cryptographic policies.
- Certificate lifecycle.
- Authentication workflow.

Establishing these design decisions before implementation made it easier to build the remaining PKI components in a structured and modular manner.

8.19 Overview

After designing the overall architecture of the Post-Quantum Public Key Infrastructure, I started implementing the individual components required to build a complete certificate-based trust framework.

Instead of relying entirely on existing certificate libraries, I implemented the core building blocks from scratch to understand how X.509 certificates are internally constructed, issued, and validated. The implementation was developed in a modular manner so that each component could be reused throughout the PKI framework.

The implementation phase was divided into several modules, beginning with the low-level encoding mechanisms and gradually progressing towards certificate generation, certificate validation, path validation, and secure communication.

8.20 DER Encoder

Objective

The first implementation step was to develop a **Distinguished Encoding Rules (DER) encoder**, which forms the foundation of X.509 certificate generation.

Since X.509 certificates are represented using **ASN.1** and serialized using **DER encoding**, understanding this encoding process was essential before implementing certificate generation.

Rather than depending on external libraries for DER encoding, I implemented the required encoding functions myself to gain a deeper understanding of how certificate data is represented internally.

Implementation

A dedicated **DEREncoder** class was developed to perform ASN.1 DER encoding.

The encoder provides reusable methods for encoding the primitive and constructed data types required during X.509 certificate construction.

The implemented encoder supports the following ASN.1 data types.

ASN.1 Type	Purpose
BOOLEAN	Represents logical values
INTEGER	Used for certificate versions, serial numbers, and other numeric fields

BIT STRING	Stores public keys and digital signatures
OCTET STRING	Stores arbitrary binary data
NULL	Represents absent parameters where required
OBJECT IDENTIFIER (OID)	Identifies cryptographic algorithms and certificate attributes
ENUMERATED	Encodes enumerated values
UTF8String	Stores Unicode text
PrintableString	Stores printable character data such as country codes
UTCTime	Represents certificate validity before the year 2050
GeneralizedTime	Represents certificate validity after the year 2050
SEQUENCE	Groups multiple ASN.1 objects into ordered structures
SET	Represents unordered collections of ASN.1 objects

In addition to these encoding methods, helper functions were implemented to correctly encode DER length fields and nested ASN.1 structures. These helper routines ensure that all certificate components follow the Distinguished Encoding Rules required by the X.509 standard.

Result

The DER encoder was successfully implemented and later reused throughout the certificate generation process.

The encoder became the foundation for constructing:

- Distinguished Names (DN)
- Subject Public Key Information
- Algorithm Identifiers
- Certificate Extensions
- TBSCertificate
- Final X.509 Certificates

Observations

Implementing the DER encoder from scratch significantly improved my understanding of how certificate data is represented before it is digitally signed.

Instead of treating certificates as opaque binary files, I learned how individual fields are converted into a standardized binary representation that can be interpreted consistently by different systems.

8.21 Utility Functions

Objective

Before implementing higher-level PKI components, I developed a collection of utility functions and helper classes that support certificate generation, inspection, and lifecycle management.

These utilities reduce code duplication and provide reusable functionality across the entire PKI implementation.

Implementation

Several helper components were implemented to simplify the development of the PKI framework.

Certificate Revocation Reasons

An enumeration named **RevocationReason** was implemented based on the reasons defined in **RFC 5280**.

The supported revocation reasons include:

- Unspecified
- Key Compromise
- CA Compromise
- Affiliation Changed
- Superseded
- Cessation of Operation
- Certificate Hold
- Remove From CRL
- Privilege Withdrawn

- Attribute Authority Compromise

Using an enumeration makes revocation handling more structured and avoids the use of arbitrary numeric values within the implementation.

Parameter Encoding

A dedicated enumeration was introduced to represent how algorithm parameters are encoded inside the X.509 **AlgorithmIdentifier** structure.

The implementation supports three parameter encoding modes:

- **ABSENT** – No parameters are encoded.
- **NULL** – A NULL parameter is encoded.
- **CUSTOM** – Custom parameter encoding is supported.

This abstraction simplifies support for different cryptographic algorithms, as each algorithm may follow different parameter encoding requirements.

Key State Management

A **KeyState** class was implemented to represent the lifecycle state of cryptographic keys.

The following key states are supported:

- ACTIVE
- ROTATED
- DESTROYED

Although only active keys are used during the current implementation, defining these states prepares the framework for future key management features such as key rotation and secure key destruction.

Serial Number Generation

A dedicated helper function was implemented to generate certificate serial numbers.

Instead of assigning sequential values, each certificate is assigned a randomly generated **159-bit serial number**.

Using random serial numbers reduces the possibility of collisions and follows modern certificate generation practices.

Entity Inspection

A utility function was also developed to inspect PKI entities during debugging and testing.

The inspection routine displays:

- Identity information
- Distinguished Name attributes
- Selected signature algorithm
- Certificate details
- Additional metadata

This utility proved useful for verifying the correctness of generated entities throughout the implementation.

Result

The utility layer successfully provided common functionality required throughout the PKI implementation.

These helper functions were reused during:

- Certificate generation
- Certificate issuance
- Certificate validation
- Revocation management
- Debugging and testing

Observations

Although these utility components are not directly involved in cryptographic operations, they significantly improved the modularity and maintainability of the implementation.

Separating common functionality into reusable helper classes also made the later implementation of certificate authorities and validation modules much cleaner.

8.22 Registries

Objective

To avoid hardcoding Object Identifiers (OIDs) and cryptographic algorithm details throughout the implementation, I developed centralized registries for certificate attributes and cryptographic algorithms.

This approach improves maintainability and makes it easier to extend the PKI with additional algorithms in the future.

Implementation

Two central registries were implemented.

Object Identifier (OID) Registry

The **OID Registry** stores the Object Identifiers associated with X.509 Distinguished Name attributes.

The implemented registry includes:

Attribute	OID
Country	2.5.4.6
State	2.5.4.8
Locality	2.5.4.7
Organization	2.5.4.10
Common Name	2.5.4.3

Each entry also specifies the appropriate ASN.1 string type to be used during DER encoding.

Algorithm Identifier Registry

A second registry was implemented to store information related to cryptographic algorithms.

For each supported algorithm, the registry maintains:

- Object Identifier (OID)
- liboqs algorithm name

- Algorithm family
- Associated hash function
- Parameter encoding method

The registry currently includes support for both classical and post-quantum algorithms, including:

- RSA
- ECDSA
- ML-DSA-44
- ML-DSA-65
- ML-DSA-87
- Falcon variants

This centralized registry allows the X.509 certificate builder to automatically generate the correct **AlgorithmIdentifier** field without hardcoding algorithm-specific values throughout the implementation.

Result

The registry implementation successfully centralized all certificate attribute identifiers and cryptographic algorithm metadata.

This made the certificate generation process significantly more modular and easier to maintain.

Observations

Implementing dedicated registries separated certificate metadata from certificate construction logic.

As a result, adding support for new post-quantum algorithms or certificate attributes requires updating only the registry instead of modifying multiple parts of the codebase.

8.23 X.509 Certificate Builder

Objective

After implementing the DER encoding layer and the required registries, the next step was to develop a framework capable of constructing complete **X.509 certificates**. The

objective of this implementation was to understand how individual certificate fields are assembled into a valid certificate structure before being digitally signed.

Instead of relying entirely on existing certificate generation libraries, I implemented a modular **X509Builder** that constructs each component of the certificate independently. This approach made it easier to understand the internal structure of X.509 certificates and allowed future support for post-quantum signature algorithms.

Implementation

The **X509Builder** class was developed to construct every major component of an X.509 certificate. Rather than generating the certificate as a single object, the builder creates each field separately before combining them into the final certificate.

The certificate construction process consists of the following stages.

Building Distinguished Names

The first step involves constructing the **Distinguished Name (DN)** for both the issuer and the subject.

Each Distinguished Name is composed of multiple attributes, including:

- Country Name (C)
- State or Province (ST)
- Locality (L)
- Organization (O)
- Organizational Unit (OU)
- Common Name (CN)
- Email Address (where applicable)

Each attribute is converted into the appropriate ASN.1 representation using the OID Registry and DER Encoder before being added to the certificate.

Algorithm Identifier Construction

Every X.509 certificate specifies the cryptographic algorithm used to generate its digital signature.

To avoid hardcoding algorithm information, the builder retrieves the required details from the **Algorithm Identifier Registry**, including:

- Object Identifier (OID)
- Hash algorithm
- Parameter encoding method

This allows different signature algorithms to be selected without modifying the certificate construction logic.

Subject Public Key Information (SPKI)

The next step involves constructing the **Subject Public Key Information (SPKI)** field.

This section contains:

- Algorithm Identifier
- Encoded public key

The SPKI field associates the subject's public key with the corresponding cryptographic algorithm and provides the information required by relying parties to verify digital signatures.

Certificate Extensions

Support for X.509 certificate extensions was incorporated into the builder to allow additional metadata to be included within the certificate.

The implemented framework provides support for commonly used extensions such as:

- Basic Constraints
- Key Usage
- Subject Key Identifier
- Authority Key Identifier

The modular design also allows new extensions to be added in future implementations with minimal changes to the existing code.

Building the TBSCertificate

Once all certificate fields have been generated, they are assembled into the **To Be Signed Certificate (TBSCertificate)** structure.

The TBSCertificate includes:

- Certificate Version
- Serial Number
- Signature Algorithm Identifier
- Issuer Distinguished Name
- Validity Period
- Subject Distinguished Name
- Subject Public Key Information
- Certificate Extensions

This structure represents the complete certificate data before the digital signature is applied.

Final Certificate Construction

After generating the TBSCertificate, the builder combines it with:

- Signature Algorithm Identifier
- Digital Signature Value

The resulting structure forms the final X.509 certificate that can be issued by a Certificate Authority.

Result

The X509Builder was successfully implemented and was capable of constructing complete X.509 certificate structures.

The implementation successfully generated:

- Distinguished Names
- Algorithm Identifiers
- Subject Public Key Information
- Certificate Extensions
- TBSCertificate
- Final certificate objects

The modular implementation allowed each component to be inspected independently before generating the complete certificate.

Observations

Implementing the X509Builder provided a much deeper understanding of how X.509 certificates are actually created internally.

Instead of viewing a certificate as a single file, I learned that it is built from several independent structures that are encoded, assembled, and finally signed to produce the complete certificate. Developing the builder in a modular manner also improved code readability and made future extensions easier to implement.

8.24 Organization Configuration

Objective

Before generating certificates, it was necessary to define the organizational information that would be embedded within every certificate issued by the PKI.

The objective of this step was to establish a consistent identity structure for all entities participating in the Post-Quantum Public Key Infrastructure.

Implementation

A configuration representing the fictional organization **QuantumSoft Pvt. Ltd.** was created.

The configuration defines common identity attributes that are shared across the PKI, including:

- Organization Name
- Country
- State
- Locality
- Organizational Units
- Root Certificate Authority
- Intermediate Certificate Authorities

This information is later used while constructing the Distinguished Names for certificates issued to employees, servers, and Certificate Authorities.

Result

The organizational configuration was successfully established and served as the common identity source throughout the certificate generation process.

All generated certificates followed a consistent naming convention and organizational hierarchy.

Observations

Defining the organizational information before certificate generation simplified the implementation considerably. Rather than repeatedly specifying identity information during certificate creation, the builder could reuse the predefined organizational configuration, ensuring consistency across the entire PKI.

8.25 Cryptographic Policy

Objective

Since the PKI was designed to support post-quantum cryptography, the next step was to define a centralized cryptographic policy that specifies the algorithms used for different security operations.

This policy provides flexibility by allowing cryptographic algorithms to be changed without modifying the certificate generation workflow.

Implementation

A cryptographic policy was defined to map different security services to their corresponding algorithms. The implemented policy includes:

Security Service	Selected Algorithm
Primary Digital Signature	ML-DSA
Experimental Digital Signature	Falcon
Backup Signature Algorithm	ML-DSA
Future Key Establishment	ML-KEM-768

By separating cryptographic policies from implementation logic, the framework can support additional algorithms in the future while maintaining the same certificate generation process.

Result

The centralized policy successfully managed the selection of cryptographic algorithms used throughout the PKI implementation.

It also prepared the framework for future migration between different post-quantum algorithms with minimal code changes.

8.26 Entity Factory

Objective

After implementing the certificate construction framework, the next step was to create a mechanism for managing the different entities that participate in the Post-Quantum Public Key Infrastructure.

The objective of this implementation was to develop a reusable framework capable of creating different PKI entities, such as Certificate Authorities, employees, and servers, while maintaining a consistent organizational structure throughout the implementation.

Rather than manually defining every entity, I implemented an **Entity Factory** that automatically creates entities with the required identity information and cryptographic configuration.

Implementation

The Entity Factory acts as a centralized component responsible for creating all entities participating in the PKI.

Each generated entity contains its identity information together with the metadata required for certificate generation.

The implemented entity structure includes:

- Entity Name
- Common Name (CN)
- Organization
- Organizational Unit

- Country
- State
- Locality
- Email Address (where applicable)
- Entity Type
- Selected Cryptographic Algorithm

Using a common factory ensures that every entity follows the same certificate structure and organizational naming conventions.

Supported Entity Types

The implemented framework supports multiple entity categories.

Entity Type	Purpose
Root Certificate Authority	Root trust anchor of the PKI
Intermediate Certificate Authority	Issues certificates on behalf of the Root CA
Employee	End-user certificates
Web Server	Server authentication certificates
Mail Server	Secure mail certificates
Future Services	Additional infrastructure components

This design allows new entity types to be introduced without modifying the certificate generation workflow.

Organization Integration

Each generated entity automatically inherits the organizational configuration defined earlier in the implementation.

This includes:

- Organization Name
- Country
- State
- Locality

- Organizational Unit

As a result, all certificates generated within the PKI maintain a consistent identity hierarchy.

Result

The Entity Factory successfully generated the entities required for the Post-Quantum PKI.

Instead of repeatedly creating entities manually, new users and servers can now be instantiated using a single reusable interface.

Observations

Implementing the Entity Factory improved the modularity of the PKI framework.

Separating entity creation from certificate generation reduced duplicate code and made it easier to extend the infrastructure as additional users and services are introduced.

8.27 Cryptographic Management Layer

Objective

After defining the entities participating in the PKI, the next step was to implement a centralized cryptographic management layer.

The objective of this module was to provide a single interface for performing all cryptographic operations used throughout the PKI framework.

Rather than directly interacting with the **liboqs** library in different parts of the project, I implemented a dedicated cryptographic management layer that abstracts the underlying cryptographic algorithms.

Implementation

The Cryptographic Management Layer provides reusable methods for performing common cryptographic operations.

The implemented functionality includes:

- Key pair generation
- Digital signature generation

- Digital signature verification
- Public key management
- Private key management
- Hash generation
- Algorithm selection

This abstraction allows higher-level PKI components to perform cryptographic operations without needing to know the implementation details of the underlying algorithms.

Supported Signature Algorithms

The implementation currently supports the following post-quantum digital signature algorithms.

Algorithm	Purpose
ML-DSA	Primary digital signature algorithm
Falcon	Experimental signature algorithm

The implementation also allows future cryptographic algorithms to be added by extending the centralized algorithm registry.

Key Generation

For every entity created by the Entity Factory, the Cryptographic Management Layer generates a corresponding public/private key pair.

The generated private key remains securely associated with the entity, while the public key is later embedded inside the X.509 certificate during certificate generation.

Digital Signature Operations

The module also provides reusable methods for:

- Signing certificate data.
- Verifying digital signatures.
- Validating message authenticity.

These operations are reused throughout the PKI implementation during certificate issuance, certificate validation, and secure communication.

Result

The Cryptographic Management Layer successfully centralized all cryptographic operations required by the PKI implementation.

Instead of repeatedly calling individual liboqs functions, higher-level modules interact with a single reusable interface.

Observations

Separating cryptographic operations into a dedicated management layer improved the overall software architecture.

The abstraction also makes the implementation more flexible because future cryptographic algorithms can be integrated without modifying the Certificate Authority or validation logic.

8.28 Integration of Entities and Cryptographic Operations

Objective

Once both the Entity Factory and Cryptographic Management Layer had been implemented, the next step was to integrate these components.

The objective of this integration was to ensure that every newly created PKI entity automatically receives its corresponding cryptographic identity.

Implementation

Whenever a new entity is created, the following workflow is executed:

1. Create the entity with its organizational information.
2. Select the configured cryptographic algorithm.
3. Generate the public/private key pair.
4. Associate the generated keys with the entity.
5. Prepare the entity for certificate generation.

This workflow ensures that every entity is fully initialized before participating in the certificate issuance process.

Result

The integration was successfully completed.

Every generated entity now contains:

- Organizational identity
- Cryptographic configuration
- Public key
- Private key
- Metadata required for X.509 certificate generation

This prepared the framework for implementing the Certificate Authority in the next stage.

Observations

Completing the integration between entity management and cryptographic operations marked an important milestone in the implementation.

At this stage, the PKI framework was capable of generating fully initialized entities with valid post-quantum key pairs. The remaining task was to establish trust by implementing the Certificate Authority responsible for issuing and managing digital certificates.

8.29 Certificate Authority

Objective

After implementing the certificate construction framework, the next step was to develop the **Certificate Authority (CA)**. The objective of this implementation was to automate the complete certificate issuance process, maintain certificate records, allocate serial numbers, and manage the lifecycle of certificates within the Post-Quantum Public Key Infrastructure.

Rather than generating certificates manually, the Certificate Authority acts as the trusted component responsible for issuing, storing, and managing digital certificates for all participating entities.

Implementation

A dedicated **CertificateAuthority** class was implemented to manage certificate-related operations throughout the PKI.

The implemented Certificate Authority performs the following responsibilities:

- Certificate issuance
- Serial number allocation
- Certificate storage
- Certificate retrieval
- Certificate revocation
- Certificate Revocation List (CRL) generation
- Certificate lifecycle management

The implementation closely follows the certificate issuance process studied during the theoretical phase while integrating the post-quantum cryptographic algorithms used throughout the project.

8.30 Certificate Issuance Process

Objective

The primary responsibility of the Certificate Authority is to issue trusted X.509 certificates.

The objective of this implementation was to generate certificates only after verifying that both the issuer and the requesting entity possess valid cryptographic identities.

Implementation

The certificate issuance process follows a structured workflow.

Step 1 – Cryptographic Identity Validation

Before issuing a certificate, the Certificate Authority verifies that both the issuing entity and the requesting entity possess valid cryptographic identities.

If either entity does not contain a valid signature identity, certificate generation is terminated and an exception is raised.

Step 2 – Serial Number Allocation

Each issued certificate is assigned a unique serial number.

The Certificate Authority generates the serial number and converts it into the required X.509 representation before certificate construction begins.

Step 3 – Distinguished Name Construction

The issuer and subject Distinguished Names are generated using the previously implemented **X509Builder**.

These names uniquely identify both the Certificate Authority issuing the certificate and the entity receiving the certificate.

Step 4 – Certificate Validity

The Certificate Authority defines the certificate validity period.

During issuance:

- The current UTC time is recorded as the issue time.
- The expiry time is calculated using the configured validity period.
- Both timestamps are encoded into the X.509 validity structure.

Step 5 – Subject Public Key Information

The subject's public key together with the associated signature algorithm is encoded into the **Subject Public Key Information (SPKI)** field.

This field binds the subject's public key with the selected post-quantum signature algorithm before certificate generation.

Step 6 – Certificate Extensions

The Certificate Authority automatically generates the default X.509 extensions required for the entity being certified.

Depending on the entity type, the generated certificate may include:

- Basic Constraints

- Key Usage
- Subject Key Identifier
- Authority Key Identifier

These extensions define the capabilities and trust relationships associated with each certificate.

Step 7 – TBSCertificate Construction

After generating every certificate component, the Certificate Authority constructs the complete **TBSCertificate**.

The TBSCertificate contains:

- Version
- Serial Number
- Issuer
- Subject
- Validity
- Subject Public Key Information
- Signature Algorithm
- Certificate Extensions

This structure represents the certificate before it is digitally signed.

Step 8 – Certificate Signing

The completed TBSCertificate is digitally signed using the issuing Certificate Authority's private key through the **CryptoManager**.

The generated digital signature proves that the certificate was issued by the trusted Certificate Authority and protects the certificate against unauthorized modification.

Step 9 – Final Certificate Construction

After generating the signature, the Certificate Authority constructs the final X.509 certificate by combining:

- TBSCertificate

- Signature Algorithm Identifier
- Digital Signature

The resulting certificate is encoded in DER format and becomes ready for distribution.

Step 10 – Certificate Storage

Once the certificate has been successfully generated, the Certificate Authority stores the certificate together with its associated metadata.

The stored certificate record includes:

- Serial Number
- Issuer
- Subject
- DER Encoded Certificate
- TBSCertificate
- Digital Signature
- Certificate Status
- Issue Time
- Expiry Time
- Subject Public Key Identifier
- Certificate Extensions
- Self-signed Status (for Root CA certificates)

The issued certificate is also associated with the subject entity, and a parent CA relationship is established for non-self-signed certificates. Additionally, the issuing CA maintains a list of all certificates it has issued.

Result

The Certificate Authority successfully automated the complete certificate issuance workflow.

Using the implemented framework, certificates could be generated, signed, stored, and associated with their respective entities without requiring manual intervention.

The implementation also maintained certificate statistics, including the total number of certificates issued by each Certificate Authority.

Observations

Implementing the Certificate Authority provided a practical understanding of how trust is established within a Public Key Infrastructure.

Rather than simply creating X.509 certificates, the Certificate Authority became the central trust anchor responsible for validating identities, assigning certificate metadata, digitally signing certificates, and maintaining certificate records throughout their lifecycle.

The modular implementation also separated certificate construction from certificate management, making the framework easier to extend and maintain.

8.31 Certificate Revocation

Objective

Issuing certificates is only one part of a Public Key Infrastructure. In real-world deployments, certificates may become invalid before their scheduled expiry due to reasons such as private key compromise, organizational changes, or termination of an entity. Therefore, the next objective was to implement a certificate revocation mechanism that allows certificates to be marked as invalid while maintaining an auditable record of the revocation.

This implementation follows the certificate revocation concepts studied during the theoretical phase and integrates them into the Post-Quantum PKI framework.

Implementation

A dedicated revocation mechanism was implemented inside the **CertificateAuthority** class to manage the lifecycle of issued certificates.

The revocation workflow performs the following operations:

1. Retrieve the certificate associated with the selected entity.
2. Determine the issuing Certificate Authority.
3. Verify that the certificate has not already been revoked.
4. Update the certificate status.

5. Record the revocation timestamp.
6. Store the revocation reason.
7. Update the Certificate Authority statistics.

If an attempt is made to revoke an already revoked certificate, the implementation raises an exception to prevent duplicate revocation operations.

Revocation Reasons

To make certificate revocation more structured, the implementation uses the **RevocationReason** enumeration developed earlier in the project.

The supported reasons include:

- Unspecified
- Key Compromise
- CA Compromise
- Affiliation Changed
- Superseded
- Cessation of Operation
- Certificate Hold
- Remove From CRL
- Privilege Withdrawn
- Attribute Authority Compromise

Using predefined revocation reasons improves consistency throughout the PKI and aligns the implementation with common X.509 certificate management practices.

Certificate Status Update

When a certificate is revoked, the following fields are updated inside the certificate record:

Certificate Field	Updated Value
Status	REVOKED
Revocation Date	Current UTC Timestamp
Revocation Reason	Selected RevocationReason

The issuing Certificate Authority also updates its internal statistics by increasing the number of revoked certificates.

Result

The certificate revocation mechanism was successfully implemented.

The framework can now invalidate certificates before their expiration date while preserving complete revocation information, including the revocation date and the reason for revocation.

Observations

Implementing certificate revocation highlighted that certificate management does not end once a certificate has been issued.

A Certificate Authority must also maintain the validity of issued certificates throughout their lifecycle by ensuring that compromised or obsolete certificates are no longer trusted.

8.32 Certificate Revocation List (CRL)

Objective

After implementing certificate revocation, the next step was to provide a mechanism through which relying parties can identify revoked certificates.

The objective of this implementation was to generate an **X.509 Certificate Revocation List (CRL)** containing all certificates revoked by a Certificate Authority.

This follows the CRL-based revocation mechanism studied during the theoretical phase.

Implementation

A dedicated **issue_crl()** function was implemented inside the Certificate Authority.

The CRL generation workflow consists of the following steps.

Step 1 – Validate the Certificate Authority

Before generating a CRL, the implementation verifies that the issuing Certificate Authority possesses a valid cryptographic identity.

If no valid signing identity exists, CRL generation is terminated.

Step 2 – Collect Revoked Certificates

The implementation retrieves every revoked certificate issued by the Certificate Authority.

This is performed using the **list_revoked_certificates()** function, which scans the list of issued certificates and returns only those whose status is marked as **REVOKED**.

Step 3 – Construct the TBSCertList

Once the revoked certificates have been collected, the Certificate Authority constructs the **To Be Signed Certificate Revocation List (TBSCertList)**.

The generated structure contains:

- CRL Version
- Issuer Distinguished Name
- Signature Algorithm
- This Update
- Next Update
- Revoked Certificate Entries

These fields represent the complete CRL before it is digitally signed.

Step 4 – Digital Signature

The generated TBSCertList is digitally signed using the private key of the issuing Certificate Authority.

The signature guarantees that the CRL originates from a trusted Certificate Authority and has not been modified after publication.

Step 5 – Final CRL Construction

After generating the digital signature, the implementation combines:

- TBSCertList
- Signature Algorithm
- Digital Signature

to construct the final DER-encoded Certificate Revocation List.

Step 6 – Store the Generated CRL

After successful generation, the CRL is stored together with its metadata.

Each generated CRL record contains:

Field	Description
Issuer	Name of the issuing Certificate Authority
This Update	CRL generation time
Next Update	Scheduled next CRL generation time
Signature Algorithm	Algorithm used to sign the CRL
Digital Signature	Signature generated by the CA
Revoked Certificate Count	Number of revoked certificates included
CRL	DER encoded Certificate Revocation List

The Certificate Authority also maintains statistics regarding the total number of CRLs generated.

Result

The Certificate Revocation List generation process was successfully implemented.

The framework can now generate signed CRLs containing all revoked certificates issued by a Certificate Authority.

The generated CRLs are stored for later distribution and verification by relying parties.

Observations

Implementing CRL generation provided a practical understanding of how certificate revocation information is distributed in real Public Key Infrastructures.

Rather than checking certificate validity individually, relying parties can consult a single digitally signed CRL to determine whether any certificate issued by a Certificate Authority has been revoked.

8.33 Certificate Revocation Management

Objective

To simplify revocation management, several helper functions were implemented to retrieve, inspect, and verify certificate revocation information.

These functions support both testing and future certificate validation modules.

Implementation

The revocation management module provides the following functionality:

Certificate Retrieval

A function was implemented to retrieve a certificate using its serial number.

This enables efficient access to issued certificates during validation and management operations.

Revoked Certificate Listing

The implementation maintains a function that returns every revoked certificate issued by a Certificate Authority.

Only certificates whose status is **REVOKED** are included in the returned list.

Revocation Status Checking

A dedicated function determines whether an entity's certificate has been revoked.

The function simply checks the certificate status and returns a Boolean result indicating whether the certificate remains valid.

Latest CRL Retrieval

To simplify certificate validation, another helper function retrieves the most recently generated CRL for a Certificate Authority.

If no CRL has been generated, the function returns **None**.

Revocation Display

For debugging and testing purposes, a display routine was implemented.

The display includes:

- Common Name
- Entity ID
- Serial Number
- Issuer
- Revocation Date
- Revocation Reason

This utility proved useful while validating the revocation workflow during implementation.

Result

The revocation management module successfully provided reusable functionality for:

- Retrieving certificates.
- Managing revoked certificates.
- Checking revocation status.
- Accessing the latest CRL.
- Displaying revocation information.

These functions prepare the framework for the certificate validation stage, where certificate status becomes an important part of the trust verification process.

Observations

Separating revocation management into reusable helper functions improved the maintainability of the Certificate Authority implementation.

Rather than embedding revocation logic inside the validation routines, the framework centralizes certificate lifecycle management within the Certificate Authority, making future extensions easier to implement.

8.34 X.509 Certificate Validation

Objective

After implementing certificate issuance and lifecycle management, the next step was to develop a certificate validation mechanism. The objective of this implementation was to verify that issued certificates are authentic, have not expired, have not been revoked, and satisfy the trust requirements of the Post-Quantum Public Key Infrastructure.

Rather than relying on external libraries, a dedicated validation component was implemented to perform the required verification checks before a certificate is accepted.

Implementation

A dedicated **X509CertificateValidator** class was implemented to validate certificates generated by the Certificate Authority.

The validator performs multiple verification steps before declaring a certificate as trusted. Each validation stage checks a different aspect of the certificate to ensure that it remains valid throughout its lifecycle.

Signature Validation

The first validation step verifies the digital signature present in the certificate.

The validator extracts the **TBSCertificate**, the **Signature Algorithm Identifier**, and the **Digital Signature** from the certificate. The issuer's public key is then used to verify the signature.

If the signature verification succeeds, the certificate is confirmed to have been issued by the claimed Certificate Authority and the certificate contents are considered unmodified.

If signature verification fails, the certificate is immediately rejected.

Validity Period Verification

The validator checks the certificate validity period by comparing the current system time with the **Not Before** and **Not After** timestamps stored inside the certificate.

The certificate is considered valid only if the current time falls within the specified validity interval.

Certificates that have expired or are not yet valid are rejected.

Revocation Status Verification

Before trusting a certificate, the validator checks whether it has been revoked.

This verification is performed using the certificate status maintained by the issuing Certificate Authority together with the generated Certificate Revocation List (CRL).

If the certificate appears in the revocation records, validation fails regardless of the remaining validity period.

Subject Key Identifier Verification

The validator also verifies the **Subject Key Identifier (SKI)** extension.

The SKI stored inside the certificate is compared with the identifier generated from the subject's public key.

This verification ensures that the certificate is correctly associated with the expected public key and helps detect inconsistencies during certificate processing.

Authority Key Identifier Verification

The validator verifies the **Authority Key Identifier (AKI)** extension whenever applicable.

The Authority Key Identifier is compared with the Subject Key Identifier of the issuing Certificate Authority.

This verification confirms that the certificate was issued by the expected Certificate Authority and maintains the integrity of the certificate hierarchy.

Certificate Extension Validation

The validator processes important X.509 extensions contained within the certificate.

The implementation validates extensions such as:

- Basic Constraints
- Key Usage
- Subject Key Identifier
- Authority Key Identifier

These extensions define the intended role and permitted usage of the certificate within the PKI.

Certificate Status Determination

Only after all verification stages complete successfully is the certificate considered valid.

The validator returns a structured validation result indicating:

- Validation Status
- Validation Errors (if any)
- Certificate Information
- Verification Outcome

This result can later be reused by higher-level PKI components such as the Path Validator and Communication Manager.

Result

The X.509 Certificate Validator was successfully implemented.

The validator correctly verified:

- Certificate digital signatures.
- Certificate validity periods.
- Certificate revocation status.
- Subject Key Identifier consistency.
- Authority Key Identifier relationships.
- Certificate extensions.

Certificates satisfying every validation requirement were successfully accepted, while invalid certificates were rejected.

Observations

Implementing the certificate validator demonstrated that certificate trust depends on far more than the digital signature alone.

A valid certificate must satisfy several independent security requirements, including signature verification, validity period verification, extension validation, and revocation status checks.

Developing the validator also established the foundation required for implementing certificate chain validation in the next stage.

8.35 Validation Workflow

To simplify certificate processing, the validator performs the verification stages in a predefined order.

The implemented workflow is illustrated below.

Receive Certificate

    |

Verify Digital Signature

    |

Check Validity Period

    |

Check Revocation Status

    |

Verify Subject Key Identifier

    |

Verify Authority Key Identifier

    |

Validate Certificate Extensions

    |

Certificate Accepted / Rejected

Executing the verification stages in this sequence allows invalid certificates to be rejected as early as possible, reducing unnecessary processing while maintaining a structured validation procedure.

8.36 Validation Outcomes

The validator can produce one of two outcomes.

Successful Validation

A certificate is accepted when:

- The digital signature is valid.
- The certificate is within its validity period.
- The certificate has not been revoked.
- The Subject Key Identifier matches the associated public key.
- The Authority Key Identifier matches the issuing Certificate Authority.
- All mandatory certificate extensions are valid.

Failed Validation

A certificate is rejected if any of the following conditions are detected:

- Invalid digital signature.
- Expired certificate.
- Certificate not yet valid.
- Revoked certificate.
- Incorrect Subject Key Identifier.
- Incorrect Authority Key Identifier.
- Invalid certificate extensions.

Providing explicit validation outcomes simplifies debugging and improves the maintainability of the PKI framework.

8.37 Certificate Path Validation

Objective

Although an individual certificate may be valid, it cannot be trusted unless the entire certification path leading to a trusted Root Certificate Authority is also valid. Therefore, the next objective was to implement a **Certificate Path Validation** mechanism capable of verifying the complete trust chain from an end entity to the trusted Root CA.

The purpose of this implementation was to ensure that every certificate presented during communication belongs to a valid certification hierarchy and that trust can be established through the issuing Certificate Authorities.

Implementation

A dedicated **PathValidator** class was implemented to verify complete certificate chains within the Post-Quantum Public Key Infrastructure.

Rather than validating certificates individually, the validator reconstructs the certification path and performs validation on every certificate in the chain before establishing trust.

The validation process follows the same hierarchical trust model used in real-world Public Key Infrastructures.

8.38 Certificate Chain Construction

Objective

Before validating a certificate chain, the complete certification path must first be reconstructed.

The objective of this stage was to identify every Certificate Authority involved in issuing the presented certificate.

Implementation

Starting from the end entity certificate, the validator repeatedly follows the issuer relationship until it reaches the trusted Root Certificate Authority.

For the implemented PKI, a typical certificate chain follows the structure shown below.

End Entity Certificate

|

Intermediate Certificate Authority

|

QuantumSoft Root Certificate Authority

Each certificate in this hierarchy becomes part of the certification path that will later be validated.

This reconstruction process ensures that no intermediate certificates are omitted during trust verification.

8.39 Trust Anchor Verification

Objective

The Root Certificate Authority acts as the trust anchor of the entire PKI.

The objective of this stage was to verify that the certification path terminates at a trusted Root Certificate Authority.

Implementation

The validator first determines whether the final certificate in the chain corresponds to a trusted Root CA.

The Root Certificate Authority must satisfy the following conditions:

- The certificate must be self-signed.
- The Root CA must exist in the trusted root store.
- The Root certificate must be valid.
- The Root certificate must not be revoked.

If the Root CA cannot be trusted, the entire certification path is rejected.

Result

The implemented validator successfully verified the trust anchor before continuing with the remaining validation stages.

Only certification paths terminating at trusted Root Certificate Authorities were accepted.

Observations

This validation stage demonstrated that the security of an entire Public Key Infrastructure ultimately depends on the trustworthiness of its Root Certificate Authority.

8.40 Certificate Chain Validation

Objective

After establishing a trusted Root Certificate Authority, the next objective was to validate every certificate contained within the certification path.

Implementation

Each certificate in the certification path is validated individually using the previously implemented **X509CertificateValidator**.

For every certificate, the validator performs:

- Signature Verification
- Validity Period Verification
- Revocation Status Verification
- Subject Key Identifier Verification
- Authority Key Identifier Verification
- Certificate Extension Validation

The certificates are processed sequentially from the Root Certificate Authority down to the end entity.

If any certificate fails validation, the entire certification path is rejected.

Result

The implementation successfully validated every certificate contained within the certification path before establishing trust.

This ensured that trust could only be established if every Certificate Authority in the hierarchy remained valid.

Observations

Validating each certificate individually prevents compromised intermediate Certificate Authorities from being trusted simply because the end entity certificate appears valid.

8.41 Parent–Child Relationship Verification

Objective

A valid certification path requires every certificate to be issued by its immediate parent Certificate Authority.

The objective of this stage was to verify that every issuer-subject relationship within the chain is correct.

Implementation

The validator compares:

- Subject Distinguished Name
- Issuer Distinguished Name
- Authority Key Identifier
- Subject Key Identifier

between consecutive certificates.

The following relationships are verified:

- End Entity ← Intermediate CA
- Intermediate CA ← Root CA

Only if every parent-child relationship is valid is the certification path accepted.

Result

The implemented validator successfully verified every parent-child relationship contained within the certificate chain.

Incorrect issuer relationships caused immediate validation failure.

Observations

This stage demonstrated that certificate trust depends not only on valid signatures but also on maintaining a consistent certification hierarchy.

8.42 Complete Path Validation Workflow

The implemented Path Validator performs certificate chain validation using the following workflow.

Receive End Entity Certificate

|

Build Certificate Chain

|

Verify Trusted Root CA

|

Validate Root Certificate

|

Validate Intermediate Certificate(s)

|

Validate End Entity Certificate

|

Verify Parent–Child Relationships

|

Certificate Chain Accepted / Rejected

This workflow closely resembles the certificate path validation process performed during TLS certificate verification.

8.43 Result

The **PathValidator** was successfully implemented.

The validator successfully:

- Constructed certificate chains.
- Verified trusted Root Certificate Authorities.

- Validated every certificate within the certification path.
- Verified issuer-subject relationships.
- Rejected incomplete or invalid certification paths.
- Established trust only after successful chain validation.

8.44 Observations

Implementing certificate path validation provided one of the clearest insights into how trust is established within a Public Key Infrastructure.

Unlike validating an individual certificate, path validation considers the entire certification hierarchy. A certificate is trusted only if every Certificate Authority in the chain is itself trusted and valid.

This implementation also demonstrated why Root Certificate Authorities are treated as trust anchors. Every certificate ultimately derives its authenticity from the Root CA, making it the foundation of the entire PKI.

8.45 PKI Framework

Objective

After implementing the individual components of the Post-Quantum Public Key Infrastructure, the next step was to integrate them into a unified framework. The objective of this implementation was to develop a central **PKIFramework** responsible for coordinating certificate management, validation, trust establishment, and entity registration throughout the system.

Rather than requiring applications to interact directly with individual components such as the Certificate Authority or Path Validator, the PKI Framework provides a single interface that manages the complete certificate lifecycle.

Implementation

A dedicated **PKIFramework** class was implemented to integrate all previously developed PKI components into a single operational framework.

The framework coordinates interactions between:

- Certificate Authority
- X.509 Certificate Builder

- Certificate Validator
- Path Validator
- Entity Factory
- Cryptographic Management Layer
- Certificate Repository

By centralizing these operations, the framework simplifies certificate management and reduces coupling between different modules.

Entity Registration

The framework provides a registration mechanism for all participating entities.

During registration:

1. A new entity is created.
2. A post-quantum key pair is generated.
3. A certificate is issued by the appropriate Certificate Authority.
4. The issued certificate is associated with the entity.
5. The entity is added to the PKI repository.

This process ensures that every participating entity possesses both a cryptographic identity and a trusted certificate before communication begins.

Certificate Management

The PKI Framework also manages certificate-related operations throughout their lifecycle.

The implemented functionality includes:

- Certificate issuance
- Certificate retrieval
- Certificate renewal (future extension)
- Certificate revocation
- Certificate lookup
- Certificate repository management

This centralized management ensures that all certificate operations follow a consistent workflow.

Trust Management

A dedicated trust management mechanism was incorporated into the framework.

The framework maintains:

- Trusted Root Certificate Authorities
- Intermediate Certificate Authorities
- Certificate chains
- Trust anchors

Before any certificate is accepted, the framework verifies that the certificate can be traced back to a trusted Root Certificate Authority through a valid certification path.

Certificate Validation Integration

Rather than validating certificates independently, the framework integrates the **X509CertificateValidator** and **PathValidator** into a unified validation workflow.

Whenever certificate validation is requested, the framework automatically performs:

- Certificate signature verification
- Validity period verification
- Revocation status verification
- Certificate extension validation
- Certificate path validation
- Trust anchor verification

Only certificates satisfying every validation stage are accepted.

Result

The PKI Framework successfully integrated all certificate management components into a single reusable system.

Instead of interacting directly with individual PKI modules, applications can now perform certificate-related operations through one centralized interface.

Observations

Developing the PKI Framework significantly improved the overall architecture of the project.

Rather than treating certificate issuance, validation, and trust management as separate operations, the framework combines them into a coherent infrastructure that closely resembles real-world enterprise PKI deployments.

8.46 Communication Manager

Objective

After completing the PKI Framework, the final implementation step was to demonstrate how the developed infrastructure could be used during secure communication.

The objective of this implementation was to build a **Communication Manager** capable of performing authenticated communication between entities using the certificates issued by the Post-Quantum Public Key Infrastructure.

Unlike previous implementations that focused only on certificate generation or validation, this module demonstrates how certificates are actually used during communication.

Implementation

A dedicated **CommunicationManager** class was implemented to coordinate secure communication between participating entities.

Before any message exchange takes place, the Communication Manager verifies the identity and trustworthiness of both communicating parties using the PKI Framework.

The communication workflow consists of several stages.

Stage 1 – Certificate Retrieval

The sender retrieves the recipient's certificate from the PKI repository.

Similarly, the recipient possesses the sender's certificate, allowing both parties to verify each other's identities.

Stage 2 – Certificate Validation

Before accepting a communication request, the Communication Manager requests the PKI Framework to validate the sender's certificate.

This validation includes:

- Certificate signature verification
- Validity period verification
- Revocation status verification
- Certificate path validation

Only certificates successfully validated by the PKI Framework are accepted.

Stage 3 – Digital Signature Generation

Once the sender has been authenticated, the outgoing message is digitally signed using the sender's private key.

The generated signature is transmitted together with the original message.

This signature guarantees:

- Sender authentication
- Message integrity
- Non-repudiation

Stage 4 – Signature Verification

After receiving the message, the recipient verifies the digital signature using the sender's public key obtained from the validated X.509 certificate.

If signature verification succeeds, the message is accepted.

Otherwise, communication is terminated.

Stage 5 – Message Authentication

Only after both certificate validation and signature verification succeed is the received message considered authentic.

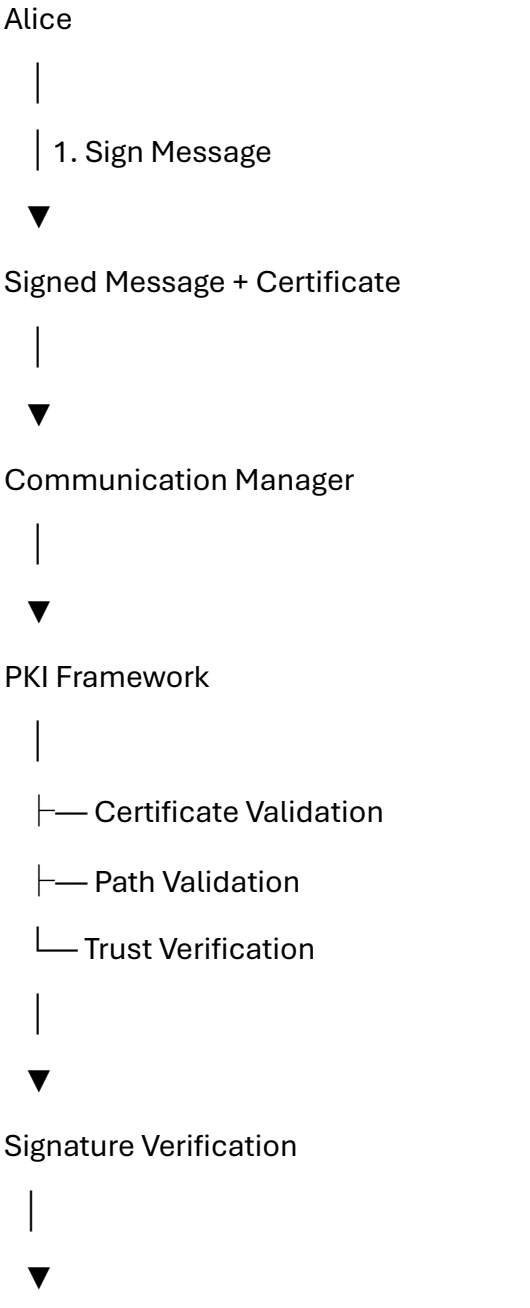
This two-stage verification process ensures that:

- The sender possesses a trusted certificate.

- The received message has not been modified during transmission.

Secure Communication Workflow

The implemented communication workflow is illustrated below.



This workflow demonstrates how multiple PKI components collaborate to establish trust before accepting any communication.

Result

The Communication Manager successfully demonstrated secure communication between entities using the developed Post-Quantum Public Key Infrastructure.

The implementation successfully:

- Retrieved entity certificates.
- Validated certificate authenticity.
- Verified certificate chains.
- Authenticated communicating entities.
- Verified digital signatures.
- Successfully accepted valid messages.

Observations

Implementing the Communication Manager demonstrated how the individual components of the PKI work together during real communication.

Rather than treating certificates as static objects, this implementation showed their practical role in establishing trust, authenticating users, and protecting message integrity before any application data is accepted.

It also highlighted the importance of validating both the certificate and the digital signature, ensuring that communication is accepted only when both the sender's identity and the transmitted message are verified.

8.47 Integration of the Complete PQC-PKI

At this stage, all major components of the Post-Quantum Public Key Infrastructure had been successfully integrated into a single operational framework.

The implemented system now supports:

- Post-quantum key generation.
- X.509 certificate construction.
- Certificate issuance.
- Certificate repository management.
- Certificate revocation.
- Certificate Revocation List generation.

- Certificate validation.
- Certificate path validation.
- Trust management.
- Secure authenticated communication.

The integration of these components transformed the project from a collection of cryptographic implementations into a functional Post-Quantum Public Key Infrastructure capable of supporting secure certificate-based communication.

8.48 Experimental Evaluation of the PQC-PKI Framework

Objective

After completing the implementation of the Post-Quantum Public Key Infrastructure, the next step was to evaluate the functionality of each major component. The objective of these experiments was to verify that the implemented framework could successfully issue certificates, establish trust, validate certificate chains, support secure communication, and correctly reject invalid or untrusted entities.

Rather than evaluating individual cryptographic algorithms, these experiments focused on validating the behaviour of the complete PKI infrastructure.

8.49 Experiment 1 – Root Certificate Authority Creation

Objective

The first experiment focused on verifying that the Root Certificate Authority could be successfully created and initialized.

Experiment

The implemented framework generated the **QuantumSoft Root CA**, including:

- Root CA identity
- Post-quantum key pair
- Self-signed X.509 certificate
- Root trust anchor

The generated certificate was stored inside the trusted certificate repository.

Result

The Root Certificate Authority was successfully created.

The generated Root CA certificate became the trust anchor for all subsequent certificate issuance operations within the PKI.

Observation

The successful creation of the Root Certificate Authority established the foundation of trust for the complete Public Key Infrastructure.

8.50 Experiment 2 – Intermediate Certificate Authority Creation

Objective

The next experiment verified that Intermediate Certificate Authorities could be created and certified by the Root Certificate Authority.

Experiment

The framework successfully generated:

- Employee Intermediate CA
- Server Intermediate CA

Each Intermediate CA generated its own post-quantum key pair and received an X.509 certificate signed by the Root Certificate Authority.

Result

Both Intermediate Certificate Authorities were successfully created and added to the certification hierarchy.

Observation

The generated hierarchy closely resembled a real enterprise PKI where the Root CA delegates certificate issuance responsibilities to Intermediate Certificate Authorities.

8.51 Experiment 3 – End Entity Certificate Issuance

Objective

The third experiment evaluated certificate issuance for end entities participating in the organization.

Experiment

Certificates were generated for multiple entities, including:

- Alice
- Bob
- Web Server
- Mail Server

For each entity, the following operations were completed:

- Key pair generation
- Certificate construction
- Digital signature generation
- Certificate issuance
- Certificate repository update

Result

All end entities successfully received valid X.509 certificates issued by the appropriate Intermediate Certificate Authority.

Observation

The successful issuance of end entity certificates confirmed that the Certificate Authority implementation correctly manages certificate generation and organizational trust relationships.

8.52 Experiment 4 – Certificate Validation

Objective

The next experiment evaluated the correctness of the implemented X.509 certificate validator.

Experiment

Each generated certificate was validated using the implemented **X509CertificateValidator**.

The validator performed:

- Signature verification
- Validity period verification
- Revocation status verification
- Subject Key Identifier verification
- Authority Key Identifier verification
- Certificate extension verification

Result

Every valid certificate passed all validation stages successfully.

Observation

The validator correctly distinguished valid certificates from invalid ones before trust was established.

8.53 Experiment 5 – Certificate Path Validation

Objective

The objective of this experiment was to verify that certificate chains could be correctly reconstructed and validated.

Experiment

The **PathValidator** reconstructed the certification path from the end entity certificate to the trusted Root Certificate Authority.

The following hierarchy was validated:

End Entity

↓

Intermediate CA

↓

Root CA

Every certificate within the certification path was individually validated before trust was established.

Result

Certificate path validation completed successfully.

The implemented framework successfully verified:

- Root trust anchor
- Intermediate Certificate Authority
- End Entity certificate

Observation

This experiment demonstrated that trust within the PKI depends on the successful validation of the complete certification hierarchy rather than an individual certificate.

8.54 Experiment 6 – Secure Communication

Objective

The next experiment evaluated the complete communication workflow implemented using the Post-Quantum Public Key Infrastructure.

Experiment

A secure communication session was established between communicating entities.

The communication workflow included:

- Certificate retrieval
- Certificate validation
- Certificate path validation

- Digital signature generation
- Digital signature verification
- Message authentication

Result

The communication completed successfully.

The recipient successfully:

- Verified the sender's certificate.
- Verified the sender's digital signature.
- Authenticated the sender.
- Accepted the transmitted message.

Observation

This experiment demonstrated that the implemented PKI can support authenticated communication using post-quantum X.509 certificates.

8.55 Experiment 7 – Certificate Revocation

Objective

This experiment evaluated the certificate revocation mechanism implemented within the Certificate Authority.

Experiment

An issued certificate was revoked using the implemented revocation mechanism.

The framework updated:

- Certificate status
- Revocation timestamp
- Revocation reason

A new Certificate Revocation List was then generated.

Result

The revoked certificate appeared correctly in the generated CRL.

Subsequent validation attempts correctly rejected the revoked certificate.

Observation

The experiment confirmed that certificate revocation information is successfully propagated throughout the PKI.

8.56 Experiment 8 – Invalid Certificate Detection

Objective

The objective of this experiment was to verify that the PKI correctly rejects invalid certificates.

Experiment

Several failure scenarios were evaluated, including:

- Modified certificate contents.
- Invalid digital signatures.
- Expired certificates.
- Revoked certificates.
- Incorrect Subject Key Identifier.
- Invalid certification path.

Result

Every invalid certificate was successfully rejected during the validation process.

Observation

The validation framework correctly identified different classes of certificate failures while preventing invalid certificates from being trusted.

8.57 Experimental Summary

The following table summarizes the outcome of all experiments conducted during the implementation.

Experiment	Expected Outcome	Actual Outcome	Status
Root CA Creation	Successful	Successful	✅
Intermediate CA Creation	Successful	Successful	✅
End Entity Certificate Issuance	Successful	Successful	✅
Certificate Validation	Valid Certificates Accepted	Successful	✅
Certificate Path Validation	Trust Chain Verified	Successful	✅
Secure Communication	Authenticated Communication	Successful	✅
Certificate Revocation	Revoked Certificates Rejected	Successful	✅
Invalid Certificate Detection	Invalid Certificates Rejected	Successful	✅

8.58 Key Findings

From the implementation and experimentation of the Post-Quantum Public Key Infrastructure, the following observations were made:

- A complete certificate-based trust infrastructure was successfully implemented using post-quantum cryptographic algorithms.
- X.509 certificates were successfully generated using the custom certificate construction framework developed during the implementation.
- The Certificate Authority successfully managed certificate issuance, storage, and lifecycle operations.
- Certificate validation correctly verified signatures, validity periods, certificate extensions, and revocation status before establishing trust.
- Certificate path validation successfully reconstructed and verified the complete certification hierarchy from end entities to the trusted Root Certificate Authority.

- The implemented Communication Manager successfully demonstrated authenticated communication using post-quantum certificates and digital signatures.
- Certificate revocation and CRL generation functioned correctly, ensuring that compromised certificates could no longer be trusted.
- The modular implementation made it possible to integrate certificate construction, validation, trust management, and secure communication into a unified Post-Quantum Public Key Infrastructure.

8.59 Overall Observations

Implementing the Post-Quantum Public Key Infrastructure brought together all the concepts studied throughout the internship, including post-quantum cryptographic algorithms, X.509 certificates, certificate authorities, certificate validation, and secure communication.

Unlike the earlier chapters, which focused on individual algorithms, this implementation demonstrated how those algorithms can be integrated into a complete trust infrastructure. Building the framework from low-level DER encoding through to secure communication provided a much deeper understanding of the internal workings of Public Key Infrastructure and highlighted the additional considerations involved in adapting PKI for post-quantum cryptography.

TLS 1.3 (Work in Progress)

9.1 Objective

After completing the implementation of the Post-Quantum Public Key Infrastructure, I started studying **Transport Layer Security (TLS) 1.3**. The objective of this phase is to understand how secure communication is established over networks and how post-quantum cryptographic algorithms can eventually be integrated into the TLS handshake.

At the current stage of the internship, this work is limited to the theoretical study of TLS 1.3. Practical implementation and experimentation are planned for the next phase of the project.

9.2 What I Learned

TLS (Transport Layer Security) is a cryptographic protocol used to provide secure communication over computer networks. It ensures confidentiality, integrity, and authentication between a client and a server.

During my study, I learned that TLS 1.3 introduces several improvements over TLS 1.2 by simplifying the handshake process and removing outdated cryptographic algorithms. These changes improve both security and performance.

9.3 Changes Introduced in TLS 1.3

During my study, I explored the major differences between TLS 1.2 and TLS 1.3.

Some of the important changes include:

- Removal of RC4, Triple DES, and CBC-mode cipher suites.
- Removal of the MAC-then-Encrypt construction.
- Removal of MD5 and SHA-1.
- Removal of RSA key transport.
- AES-GCM is used as the mandatory authenticated encryption algorithm.
- All key exchanges are performed using Diffie-Hellman (DH) or Elliptic Curve Diffie-Hellman (ECDH).
- Supported elliptic curves include P-256, Curve25519, and P-384.

These changes strengthen the protocol and provide better protection against modern cryptographic attacks.

9.4 Components of TLS 1.3

I studied the two main components of the TLS protocol.

Handshake Protocol

The Handshake Protocol is responsible for:

- Authenticating the server.
- Negotiating cryptographic algorithms.
- Establishing shared cryptographic keys before secure communication begins.

Record Layer Protocol

Once the handshake is complete, the Record Layer Protocol encrypts and authenticates all application data exchanged between the client and the server.

9.5 TLS 1.3 Handshake

I studied the complete TLS 1.3 handshake process and understood how a secure session is established between a client and a server.

Phase 1 – Client Hello

The client sends:

- Supported TLS versions.
- Supported cryptographic algorithms.
- A client nonce.
- A temporary ECDH public key.

This information allows the server to select compatible cryptographic parameters.

Phase 2 – Server Hello

The server:

- Generates its own temporary ECDH public key.
- Computes the shared master secret.
- Derives the required session keys.
- Sends its certificate chain.
- Sends its digital signature over the handshake transcript.

- Sends the server finished MAC.

Phase 3 – Client Verification

The client:

- Verifies the server's certificate chain.
- Verifies the server's digital signature.
- Computes the same shared master secret.
- Derives the session keys.
- Verifies the server finished MAC.
- Sends the client finished MAC to the server.

Phase 4 – Handshake Completion

The server verifies the client finished MAC.

Once this verification succeeds, both parties possess the same shared session keys and secure communication can begin.

9.6 RSA vs ECDH Key Exchange

As part of the study, I compared the traditional RSA key transport mechanism with Elliptic Curve Diffie-Hellman (ECDH).

I learned that RSA key transport, which was used in TLS 1.2, does not provide **Forward Secrecy**. If an attacker records encrypted traffic and later compromises the server's private key, the previously captured communication can also be decrypted.

In contrast, ECDH generates a fresh ephemeral key pair for every session. Even if the server's long-term private key is compromised in the future, previously established session keys cannot be recovered. This property is known as **Forward Secrecy**, which is one of the key security improvements introduced in TLS 1.3.
